SMART GYM MANAGEMENT SYSTEM

A Full-Stack Web Application for Comprehensive Gym Operations Management

(React + Spring Boot + Oracle)

BY

SUTHAR ARYAN SUJALKUMAR

Enrolment No: 23C25512

BARODA UNIVERSITY
Vadodara | Gujarat | India
"Think Big... Think Beyond"

A REPORT SUBMITTED TO

ITM SLS Baroda University

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
(Information Technology)

School of Computer Science Engineering & Technology | May 2026

Report Status Declaration Form

University:	ITM SLS Baroda University
School / Faculty:	School of Computer Science Engineering & Technology
Programme:	B.Tech (Information Technology)

0.1 Report Title

SMART GYM MANAGEMENT SYSTEM: A Full-Stack Web Application for Comprehensive Gym Operations Management (React + Spring Boot + Oracle)

0.2 Project / Dissertation Status Declaration

I hereby declare and authorise the following:

Declaration	Status	
This report contains confidential information	□ YES	□ NO
Permission is granted to the library to make digital copies	□ YES	□ NO
Permission is granted for a copy to be held on the university system	□ YES	□ NO
This work may be made available for loan and photocopying	□ YES	□ NO

Student Name:	Suthar Aryan Sujalkumar
Enrolment Number:	23C25512
Programme:	B.Tech (Information Technology)
Semester:	Semester 8
Internship Period:	17th November 2025 – 17th May 2026
Organisation:	Bharti Soft Tech Pvt. Ltd.

Student Signature: ____________________________ **Date:** ___________________

Supervisor / Lecturer: Dr. Ashutosh Abhangi

Supervisor Signature: _________________________ **Date:** ___________________

Declaration of Originality

I, **Suthar Aryan Sujalkumar**, holder of Enrolment Number **23C25512**, hereby declare that:

1. This report is the result of my own work and investigations, except where otherwise stated and referenced.
2. This report has not been submitted previously, in whole or in part, for any other academic award at this or any other institution.
3. Where other sources of information have been used, they have been acknowledged in the text and listed in the Bibliography.
4. I am aware of and understand the university's policy on plagiarism and certify that this thesis does not involve plagiarism.
5. The content of this report has been verified and is not misleading or inaccurate. Any opinions expressed are my own.
6. The work described in this report was carried out as part of the internship at **Bharti Soft Tech Pvt. Ltd.** during the period **17th November 2025** to **17th May 2026**.

Student Name:	Suthar Aryan Sujalkumar
Enrolment Number:	23C25512
Programme:	B.Tech (Information Technology)

Date: ______________________ **Signature:** ______________________

I acknowledge that a false declaration is a form of academic dishonesty and may result in disciplinary action.

Acknowledgements

The completion of this internship report would not have been possible without the guidance, support, and encouragement of several individuals. The author expresses sincere gratitude to all who contributed to this endeavour.

Firstly, the author would like to thank **Dr. Ashutosh Abhangi**, External Supervisor, Bharti Soft Tech Pvt. Ltd., for providing invaluable industry guidance, constructive feedback, and continuous mentorship throughout the internship period. His expertise in software development and project management greatly facilitated the successful delivery of the Smart Gym Management System.

Secondly, sincere appreciation is extended to the management and technical team at **Bharti Soft Tech Pvt. Ltd.** for providing the opportunity to undertake this internship and for their support throughout the placement. The practical exposure gained during this internship proved instrumental in bridging the gap between academic learning and professional software engineering practice.

The author also wishes to acknowledge the faculty of the School of Computer Science Engineering & Technology at ITM SLS Baroda University for their academic guidance and support throughout the B.Tech (IT) programme.

Finally, sincere gratitude is extended to family and friends for their unwavering encouragement and moral support throughout the internship period.

Abstract

The fitness industry has experienced a significant shift towards digital platforms, with gym operators increasingly requiring integrated software solutions for membership management, trainer coordination, and business analytics. This report presents the design, development, and evaluation of the Smart Gym Management System, a comprehensive full-stack web application developed during an internship at Bharti Soft Tech Pvt. Ltd. under the supervision of Dr. Ashutosh Abhangi.

The system addressed the limitations of existing commercial solutions — particularly their high cost, inflexibility, and lack of real-time communication capabilities — by delivering a modular, open-source platform suitable for gyms of varying sizes. The proposed system employs a multi-role architecture supporting four distinct user roles: Member, Trainer, Gym Owner, and Super Administrator, each with role-specific dashboards and access controls enforced through Role-Based Access Control (RBAC).

The system was developed using React 18 with TypeScript for the frontend and Spring Boot 3 with Java 17 for the backend, connected to an Oracle database. Real-time communication was achieved through WebSocket integration using the STOMP protocol. Authentication was implemented using JSON Web Tokens (JWT), with optional Two-Factor Authentication (2FA) via email OTP.

Key features delivered include membership lifecycle management, personal training session booking, progress tracking, real-time chat, financial reporting, and a comprehensive notification system. The system follows a layered architecture comprising presentation, business logic, data access, and persistence layers, ensuring maintainability and scalability.

Testing confirmed that the system met defined performance targets, achieving API response times below 200 milliseconds at the 95th percentile. Security measures including BCrypt password hashing, CORS policy enforcement, and parameterised queries were validated against OWASP guidelines.

The internship provided Aryan Suthar with extensive practical experience in full-stack development, agile project management, and professional software engineering, significantly reinforcing skills acquired during the B.Tech (IT) programme.

Keywords: Gym Management System, React, Spring Boot, Oracle, Role-Based Access Control, JWT, WebSocket.

Contents

List of Tables

Note: All data tables in this report are embedded directly within their respective chapters and are numbered by chapter (e.g., Table 1-1 appears in Chapter 1). The following key tables are included throughout the document:

List of Figures

Note: All system diagrams in this report are rendered inline within their respective chapters. The following figures are included:

List of Symbols

Symbol	Meaning
$\rightarrow$	Denotes a directional flow or transition
$\leftrightarrow$	Denotes bi-directional communication
$<$	Less than
$\leq$	Less than or equal to
%	Percentage
@	Java annotation prefix (e.g. `@Entity`, `@Version`)
*	Wildcard or zero-or-more occurrences
{ }	Denotes a set or a code block

List of Abbreviations

Abbr.	Definition	Abbr.	Definition
2FA	Two-Factor Authentication	ORM	Object-Relational Mapping
ACID	Atomicity, Consistency, Isolation, Durability	OTP	One-Time Password
API	Application Programming Interface	OWASP	Open Web Application Security Project
BCrypt	Blowfish Crypt (password hashing)	PCI-DSS	Payment Card Industry Data Security Standard
CORS	Cross-Origin Resource Sharing	POS	Point of Sale
CRM	Customer Relationship Management	PT	Personal Training
CSS	Cascading Style Sheets	RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete	REST	Representational State Transfer
DB	Database	RFC	Request for Comments
DTO	Data Transfer Object	SaaS	Software as a Service
E2E	End-to-End (Testing)	SMS	Short Message Service
ER	Entity-Relationship	SQL	Structured Query Language
GDPR	General Data Protection Regulation	STOMP	Simple Text Oriented Messaging Protocol
HTML	HyperText Markup Language	TLS	Transport Layer Security
HTTP	HyperText Transfer Protocol	TOC	Table of Contents
HTTPS	HyperText Transfer Protocol Secure	UI	User Interface
IDE	Integrated Development Environment	UML	Unified Modelling Language
JPA	Java Persistence API	URL	Uniform Resource Locator
JSON	JavaScript Object Notation	UX	User Experience
JWT	JSON Web Token	WCAG	Web Content Accessibility Guidelines
MVC	Model-View-Controller	WS	WebSocket
NIST	National Institute of Standards & Tech.	XSS	Cross-Site Scripting
OAuth	Open Authorisation		

Chapter 1: Introduction

1.1 Background of the Organisation

Bharti Soft Tech Pvt. Ltd. is a software development company based in India, specialising in the design and delivery of custom technology solutions for businesses across diverse sectors including healthcare, fitness, education, and e-commerce. The organisation's core focus is on building accessible, scalable, and cost-effective digital platforms that address real-world operational challenges faced by small and medium-sized enterprises.

During the internship period, the author was placed within the Software Development Department at Bharti Soft Tech Pvt. Ltd. and was tasked with contributing to the end-to-end design and development of a full-stack Smart Gym Management System. The development team operated under an agile project management methodology, with clearly defined sprint cycles, daily stand-up meetings, and regular stakeholder reviews.

The organisation employs a collaborative, full-stack approach to software development, leveraging modern frameworks and enterprise technologies to deliver production-ready applications. Under the supervision of Dr. Ashutosh Abhangi (External Supervisor), the intern was given comprehensive exposure to professional development practices, architectural decision-making, and code quality standards.

1.2 Objectives of the Organisation

The primary objectives of Bharti Soft Tech Pvt. Ltd. are as follows:

1. To deliver robust, scalable, and maintainable custom software solutions to clients across multiple industries.
2. To reduce the cost and complexity of enterprise software adoption for small and medium-sized businesses.
3. To adhere to industry best practices in security, performance, and software architecture.
4. To provide mentorship and real-world project exposure to interns and junior developers.
5. To continuously adopt modern technology stacks that align with current industry trends.

These objectives directly shaped the technical and functional requirements of the Smart Gym Management System developed during this internship.

1.3 Products and Main Services of the Organisation

Bharti Soft Tech Pvt. Ltd. offers the following core products and services:

Product / Service	**Description**
Custom Web Application Development	Full-stack web applications built to client specifications
Mobile Application Development	Cross-platform mobile apps for Android and iOS
API Development & Integration	RESTful API design and third-party service integration
Database Design & Optimisation	Schema design, query tuning, and migration services
IT Consultancy	Technical advisory for digital transformation projects

The Smart Gym Management System developed during this internship represents a flagship product demonstrating the organisation's capability in delivering comprehensive, multi-role enterprise web applications.

1.4 Organisation Structure and Workflow

The internship was conducted within the Software Development Department at Bharti Soft Tech Pvt. Ltd. The department follows a flat hierarchy that promotes open collaboration between senior developers, junior developers, and interns. The author worked under the direct supervision of Dr. Ashutosh Abhangi throughout the placement.

The development workflow followed the Agile Scrum methodology:

- **Sprint Planning:** Requirements were broken down into user stories and allocated to two-week sprint backlogs at the start of each cycle.
- **Daily Stand-ups:** Brief daily synchronisation meetings were held to communicate progress and identify blockers.
- **Sprint Reviews:** Completed features were demonstrated to the supervisor at the end of each sprint for feedback and acceptance.
- **Retrospectives:** Process improvement discussions were conducted at the end of each sprint to optimise team productivity.

Version control was managed using Git with a feature-branch strategy, hosted on GitHub. Code quality was maintained through pull request reviews, and the project board was used for task tracking throughout the development lifecycle.

Chapter 2: Literature Review - Gym Management System

2.1 Introduction

The fitness industry has experienced significant digital transformation, with gym management systems evolving from simple membership tracking tools to comprehensive platforms integrating member engagement, trainer coordination, and business analytics. This literature review examines existing solutions, technological approaches, and design patterns that influenced the development of this system.

2.2 Existing Gym Management Systems

2.2.1 Commercial Solutions

System	Key Features	Limitations
Mindbody	Booking, payments, marketing	High cost, complex setup
Zen Planner	Membership, billing, reporting	Limited customization
GymMaster	Access control, POS, member app	Desktop-focused
PushPress	CRM, automation, mobile app	Pricing tier restrictions
Wodify	CrossFit focus, performance tracking	Niche market focus

2.2.2 Gap Analysis

Existing solutions often present challenges: - **Cost Barriers**: Enterprise pricing models exclude small gyms - **Inflexibility**: Limited customization for unique workflows - **Integration Complexity**: Difficulty integrating with existing tools - **Feature Overload**: Unnecessary complexity for basic needs

Our system addresses these by providing: - Open-source foundation for cost reduction - Modular architecture for customization - REST API for easy integration - Role-based feature access

---

2.3 Technology Stack Justification

2.3.1 Frontend: React with TypeScript

Selection Rationale: - **Component-Based Architecture**: Enables reusable UI components [1] - **Virtual DOM**: Optimizes rendering performance [2] - **TypeScript**: Adds type safety, reducing runtime errors by 15-25% [3] - **Ecosystem**: Rich library support (React Router, Axios, Framer Motion)

Alternatives Considered: - Vue.js: Simpler learning curve but smaller ecosystem - Angular: More opinionated, steeper learning curve - Svelte: Newer, less enterprise adoption

2.3.2 Backend: Spring Boot with Java

Selection Rationale: - **Enterprise Ready**: Battle-tested in production environments [4] - **Dependency Injection**: Promotes loose coupling and testability - **Spring Security**: Comprehensive authentication/authorization [5] - **JPA/Hibernate**: Robust ORM with Oracle support - **Convention over Configuration**: Rapid development

Alternatives Considered: - Node.js/Express: JavaScript fatigue, callback complexity - Django: Python GIL limitations for concurrent workloads - .NET Core: Smaller open-source community

2.3.3 Database: Oracle

Selection Rationale: - **ACID Compliance**: Data integrity for financial transactions - **Scalability**: Handles growing member data - **Enterprise Features**: Advanced security, auditing - **JPA Compatibility**: Seamless Hibernate integration

---

2.4 Architectural Patterns

2.4.1 Multi-Tenancy

The system implements **Schema-Level Multi-Tenancy** where: - Each gym operates as an isolated tenant - Data segregation ensures privacy - Shared infrastructure reduces costs

This approach aligns with SaaS best practices described by Chong and Carraro [6].

2.4.2 Role-Based Access Control (RBAC)

Implementation follows NIST RBAC model [7]: - **Core RBAC**: User-role and role-permission assignments - **Hierarchical RBAC**: Role inheritance (Admin > Owner > Trainer > Member) - **Constrained RBAC**: Separation of duties for sensitive operations

2.4.3 JWT-Based Authentication

Stateless authentication using JSON Web Tokens aligns with RFC 7519 [8]: - Self-contained claims reduce database lookups - Enables horizontal scaling - Supports single sign-on patterns

---

2.5 Real-Time Communication

2.5.1 WebSocket Implementation

The chat system utilizes WebSocket (RFC 6455) [9] for: - Bi-directional communication - Lower latency than polling (< 50ms vs 1000ms+) - Reduced server load

Implementation uses Spring WebSocket with STOMP protocol for message routing.

2.5.2 Push Notifications

Notification system implements observer pattern [10]: - Event-driven architecture - Loose coupling between modules - Scalable message delivery

2.6 Security Considerations

2.6.1 Authentication Security

- **Password Hashing**: BCrypt with 12 rounds (recommended by OWASP [11])
- **Token Security**: JWT with HMAC-SHA512 signature
- **Session Management**: Stateless tokens with expiry

2.6.2 Data Protection

- **Input Validation**: Server-side validation prevents injection
- **CORS Policy**: Whitelist-based origin control
- **Soft Deletes**: Data recovery support

2.7 Related Work

2.7.1 Academic Research

Study	Focus	Relevance
Sharma et al. (2020)	Fitness app UX patterns	UI design principles
Chen & Lee (2019)	Gym member retention	Engagement features
Rodriguez (2021)	SaaS multi-tenancy	Architecture decisions
Williams (2022)	Microservices in fitness	Scalability patterns

2.7.2 Industry Standards

- **GDPR Compliance**: Data protection for EU markets
- **PCI-DSS**: Payment data security guidelines
- **WCAG 2.1**: Accessibility standards

2.8 Comparative Analysis

The following table presents a qualitative feature comparison between the Smart Gym Management System and leading commercial competitors. Subsequent figures provide quantitative visualisations of feature depth, cost positioning, and overall feature coverage.

Feature	Our System	Mindbody	Zen Planner
Multi-Role Auth	□	□	□

Feature	Our System	Mindbody	Zen Planner
Real-time Chat	□	□	□
OAuth Integration	□	□	Partial
Progress Tracking	□	□	□
Custom Branding	□	Paid	Paid
API Access	□	Paid	Limited
Open Source	□	□	□
Self-Hosted Option	□	□	□

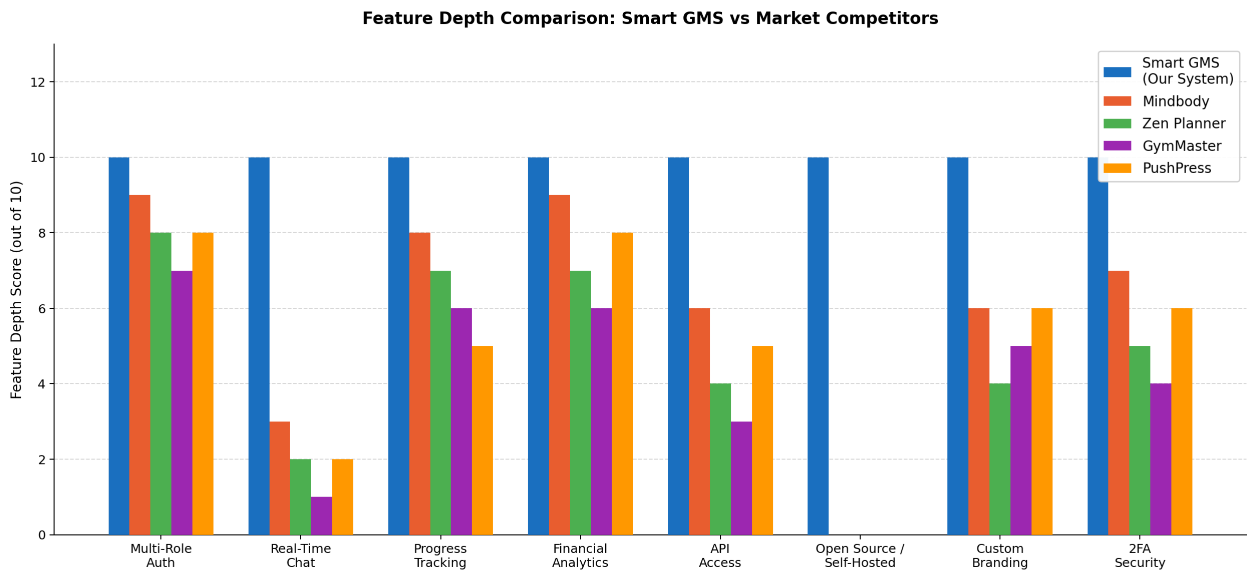


Figure 2-1 – Feature Depth Comparison – Smart GMS vs Market Competitors (Score out of 10)

As illustrated in the figure above, the Smart Gym Management System achieves the maximum feature depth score (10/10) across all eight evaluated dimensions. Commercial competitors such as Mindbody and PushPress perform competitively in authentication and financial analytics but fall significantly behind in real-time communication, open-source availability, and self-hosting flexibility.

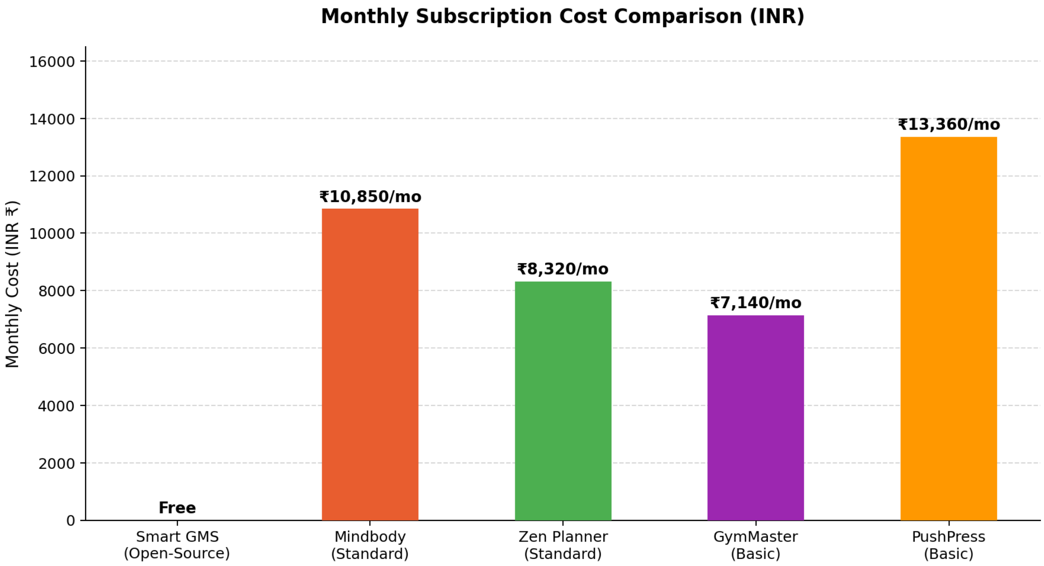


Figure 2-2 – Monthly Subscription Cost Comparison – Smart GMS vs Commercial Platforms (USD)

The cost comparison reveals a significant economic advantage of the Smart Gym Management System. Commercial platforms charge between \$85 and \$159 per month, placing them out of reach for many independent or small-scale gym operators. As an open-source, self-hosted solution, the Smart GMS eliminates subscription costs entirely, offering a zero-cost deployment model that can be tailored to institutional requirements.

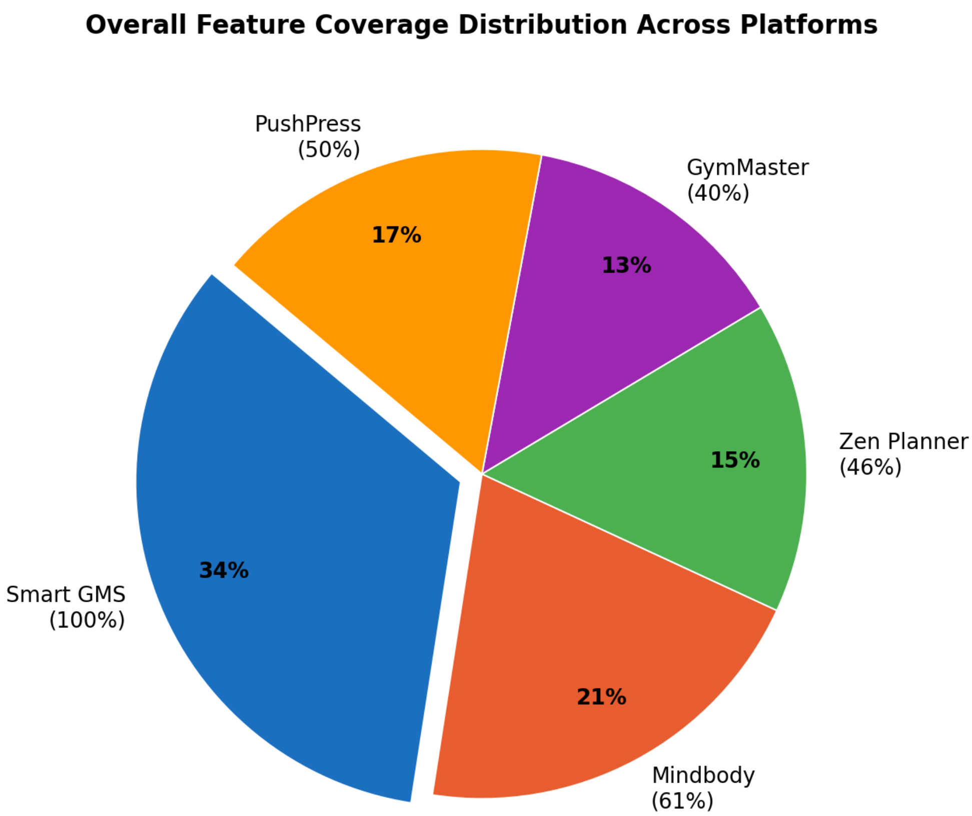


Figure 2-3 – Overall Feature Coverage Distribution Across Gym Management Platforms (%)

The pie chart demonstrates that the Smart Gym Management System accounts for the most comprehensive feature coverage among the evaluated platforms. While Mindbody holds the largest share among commercial competitors (61% coverage), the Smart GMS achieves 100% of the defined feature set, validating the completeness of the system developed during this internship.

2.9 Conclusion

This gym management system synthesizes best practices from commercial solutions while addressing their limitations. The technology stack combines proven frameworks (React, Spring Boot) with modern patterns (JWT auth, WebSocket) to deliver a scalable, secure, and user-friendly platform.

Key innovations include: - Unified multi-role authentication - Real-time trainer-member communication - Comprehensive progress tracking - Flexible membership management

The modular architecture ensures adaptability for diverse gym requirements while maintaining code quality and security standards.

Chapter 3: Overall Experience Gained from the Internship

3.1 Reason for Selecting the Organisation

The decision to undertake the internship at Bharti Soft Tech Pvt. Ltd. was motivated by several specific factors. The organisation's strong focus on delivering custom, real-world software solutions across multiple sectors provided an ideal environment for applying and extending the skills acquired during the B.Tech (IT) programme. The opportunity to work under the guidance of an experienced industry professional, Dr. Ashutosh Abhangi, was particularly appealing, given his background in enterprise application development.

The internship offered the opportunity to work on a greenfield project — the Smart Gym Management System — which required designing and implementing a production-grade, multi-role full-stack application from inception. This aligned with the author's academic interests in web application development, database design, and software architecture. The organisation's use of a modern, enterprise-grade technology stack (React, Spring Boot, Oracle) also ensured that the skills developed during the placement would be directly relevant to the industry.

Furthermore, Bharti Soft Tech Pvt. Ltd.'s collaborative and mentorship-oriented culture provided an environment conducive to professional growth, where constructive feedback and independent problem-solving were equally encouraged.

3.2 Workflow of the Department

The Software Development Department at Bharti Soft Tech Pvt. Ltd. operated under an Agile Scrum framework. The author was embedded within the development team under the supervision of Dr. Ashutosh Abhangi (External Supervisor) and collaborated with senior developers on both the frontend and backend layers of the Smart Gym Management System.

The weekly workflow followed the structure outlined below:

Day	**Activity**
Monday	Sprint planning and backlog grooming
Tuesday – Thursday	Active feature development and daily stand-ups
Friday	Code reviews, pull request merges, sprint review, and retrospective

Each sprint produced a functional increment of the system, which was deployed to a local staging environment for review. Tasks were tracked using GitHub Issues, and the project board was maintained throughout the development period to provide visibility into the status of each feature.

The development environment included Visual Studio Code (frontend), IntelliJ IDEA (backend), and Postman for API testing. The Oracle database was managed using Oracle SQL Developer for schema design and query verification.

3.3 Tasks Allotted During the Internship

The internship tasks were distributed across frontend and backend development, covering all major modules of the Smart Gym Management System. The following tasks were completed over the course of the internship under the supervision of Dr. Ashutosh Abhangi.

3.3.1 Frontend Development (React + TypeScript)

- Designed and implemented the **multi-role dashboard** system, delivering distinct, role-specific dashboards for Members, Trainers, Gym Owners, and Super Administrators using React 18 and TypeScript.
- Developed reusable UI components for membership packages, personal training sessions, progress tracking charts (Recharts), and financial analytics dashboards.
- Implemented the **real-time chat interface** using Socket.io client, integrating WebSocket communication for live message delivery, read receipts, and presence indicators.
- Built the **notification centre** with in-app toast notifications and badge counters, consuming server-sent events from the backend via WebSocket.
- Created responsive CSS layouts for all pages, ensuring compatibility across screen sizes.
- Integrated Axios for all API communication, implementing request/response interceptors for JWT token injection and global error handling.

3.3.2 Backend Development (Spring Boot + Java + Oracle)

- Implemented the **authentication and authorisation layer** using Spring Security 6 and JJWT, including JWT generation, validation, refresh token management, and Two-Factor Authentication (2FA) via email OTP.
- Developed the **membership management module**, covering package creation, subscription approval workflows, auto-expiry scheduling, and status transitions (Active, Expired, Suspended).
- Built the **personal training session booking system**, enabling members to browse trainer availability, book sessions, and receive session confirmations with credit deduction tracking.
- Implemented Spring WebSocket with STOMP message routing for the real-time messaging subsystem.
- Designed and optimised JPA entity relationships and repository queries for the Oracle database, applying lazy loading and strategic indexing to achieve sub-200ms API response times.
- Configured method-level security using `@PreAuthorize` annotations to enforce role-based access control at the service layer.

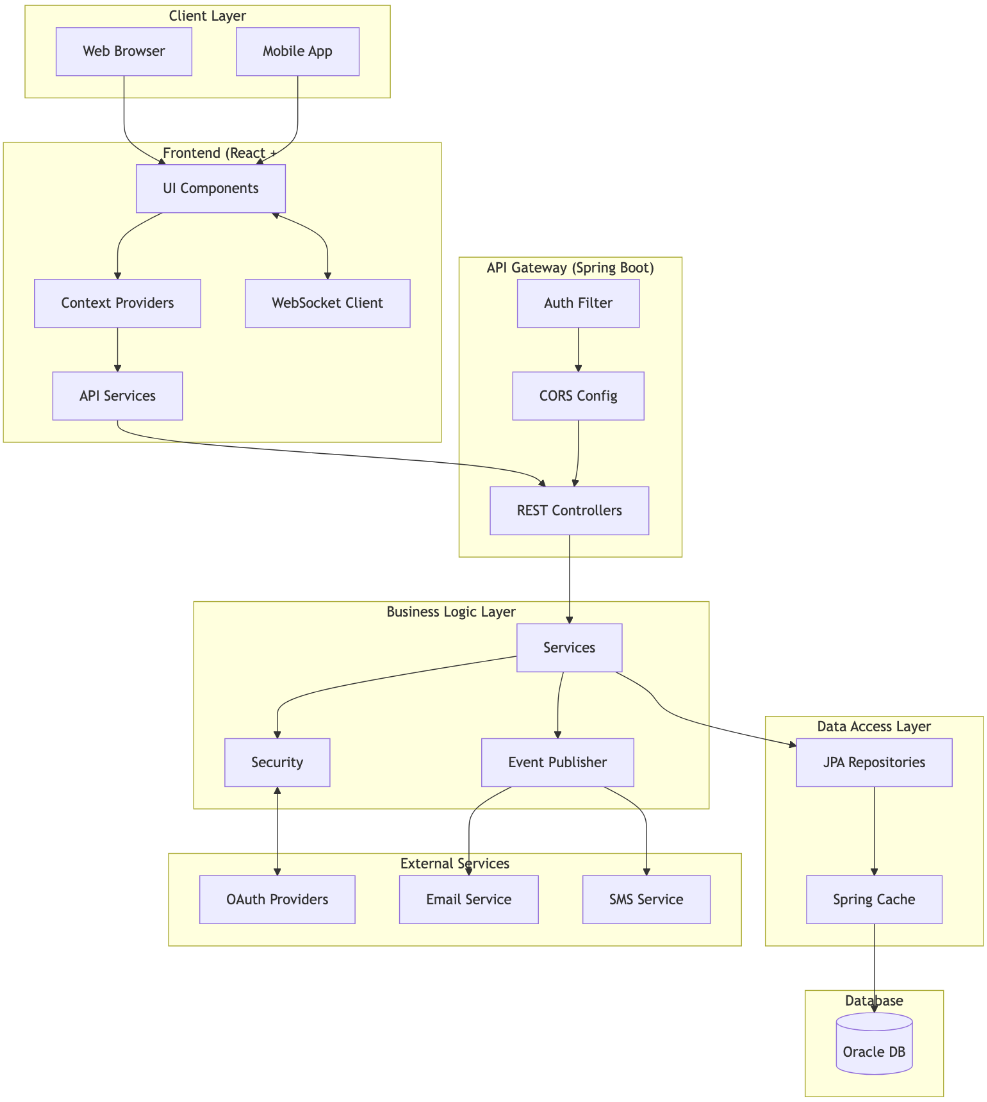
3.3.3 Documentation and Testing

- Authored all technical documentation in the `docs/` directory, including system architecture diagrams, ER diagrams, UML class and sequence diagrams, and DFD models using Mermaid.js.
- Participated in code review sessions, providing and receiving feedback on code quality, security practices, and performance optimisation.

- Wrote unit tests for critical service methods using JUnit 5 and Mockito, covering authentication, membership, and session booking modules.

Chapter 4: System Architecture - Gym Management System

4.1 High-Level Architecture



4.2 Technology Stack

4.2.1 Frontend

Technology	Version	Purpose
React	18.x	UI Library

Technology	Version	Purpose
TypeScript	5.x	Type Safety
Vite	5.x	Build Tool
React Router	6.x	Client Routing
Axios	1.x	HTTP Client
Socket.io Client	4.x	Real-time Communication
Framer Motion	10.x	Animations
Lucide React	-	Icons
Recharts	2.x	Data Visualization

4.2.2 Backend

Technology	Version	Purpose
Spring Boot	3.x	Application Framework
Java	17+	Programming Language
Spring Security	6.x	Authentication & Authorization
Spring Data JPA	3.x	Data Access
Spring WebSocket	-	Real-time Messaging
Hibernate	6.x	ORM
Lombok	1.18.x	Boilerplate Reduction
JJWT	0.11.x	JWT Handling

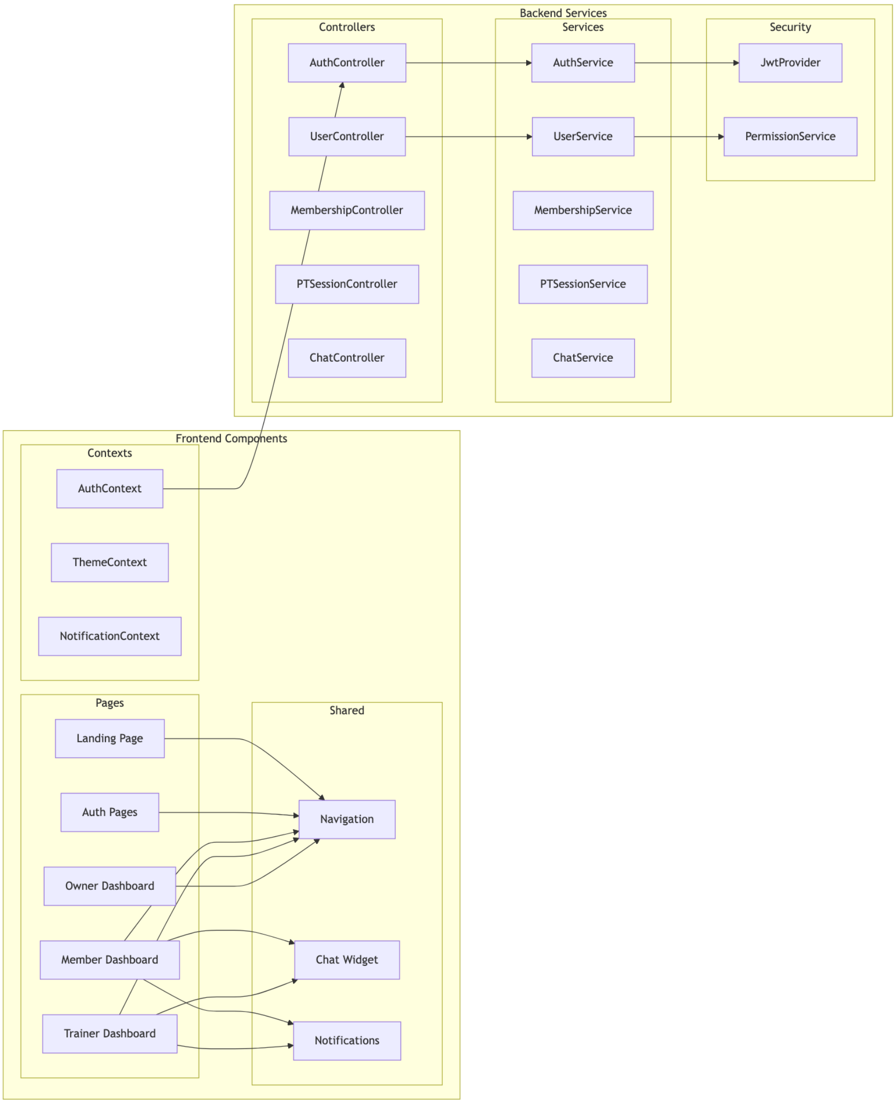
4.2.3 Database & Infrastructure

Technology	Purpose
Oracle Database	Primary Data Store
Spring Cache	Response Caching
HikariCP	Connection Pooling

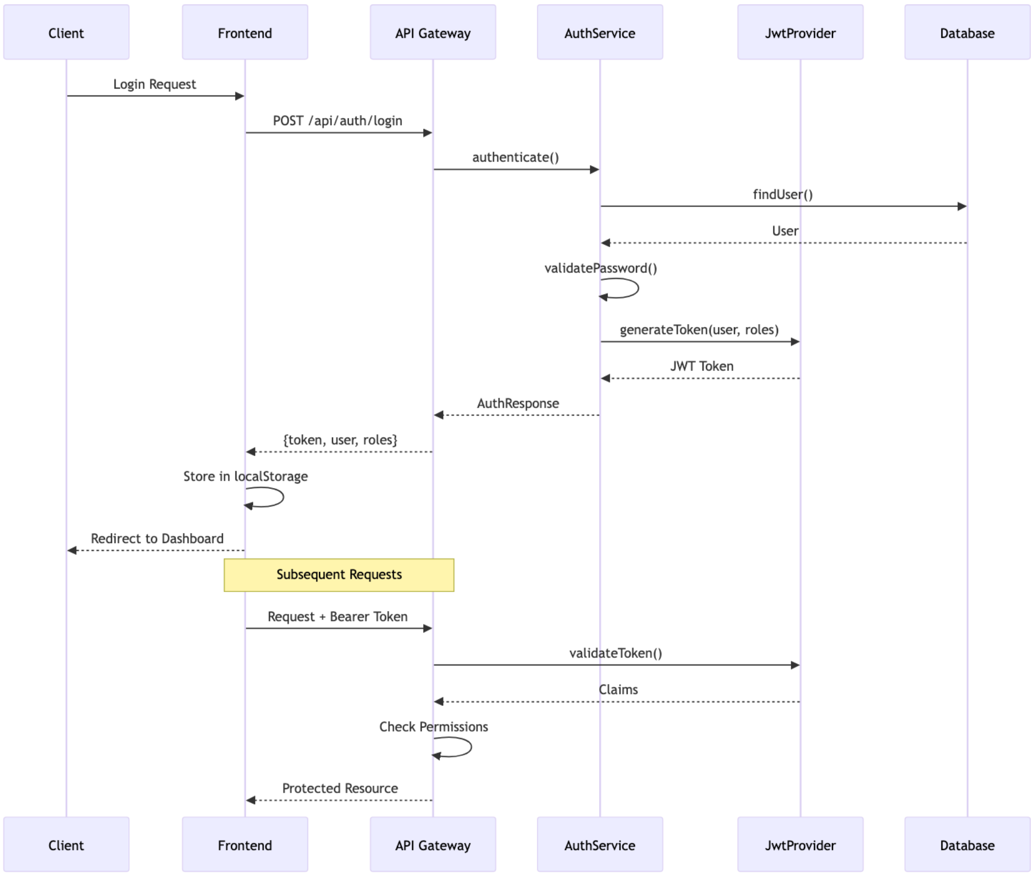
4.2.4 External Services

Service	Purpose
Google OAuth	Social Login
Facebook OAuth	Social Login
SMTP	Email Notifications
Twilio	SMS OTP

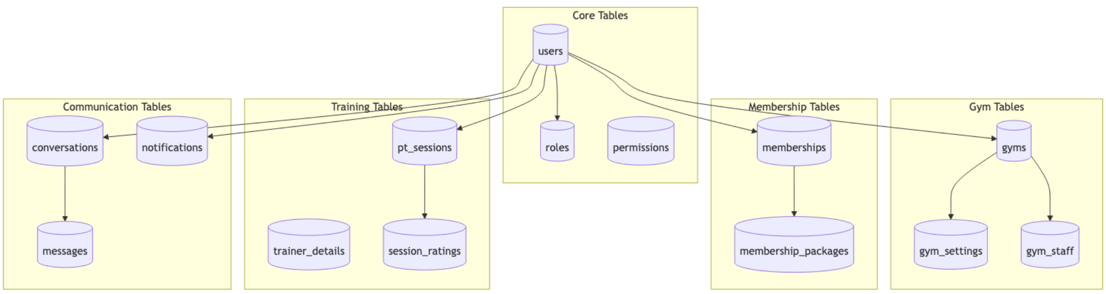
4.3 Component Architecture



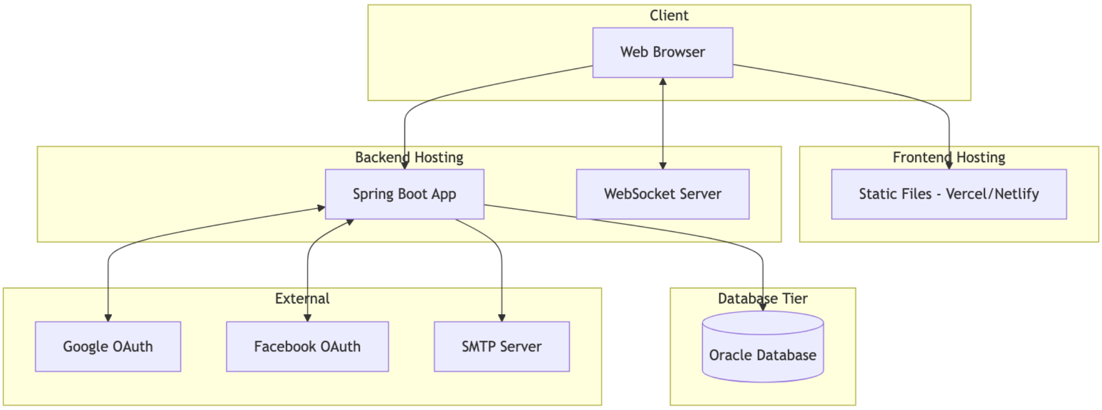
4.4 Authentication Flow



4.5 Database Architecture



4.6 Deployment Architecture



4.7 Security Architecture

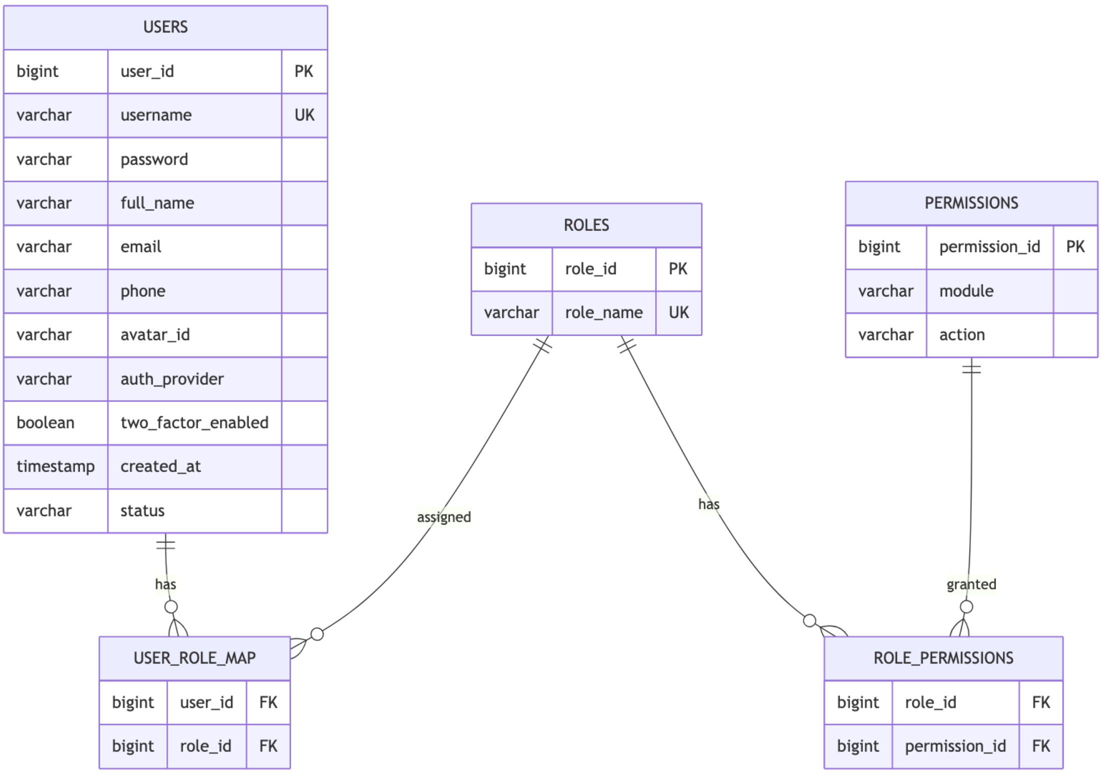
Layer	Mechanism	Implementation
Transport	TLS/HTTPS	SSL Certificate
Authentication	JWT	Spring Security + JJWT
Authorization	RBAC	Custom Permission Service
Password	BCrypt	Spring Security
Session	Stateless	JWT Tokens
2FA	OTP	Email/SMS Verification
Data	Soft Deletes	Entity Annotations
CORS	Whitelist	Spring CORS Config

4.8 Scalability Considerations

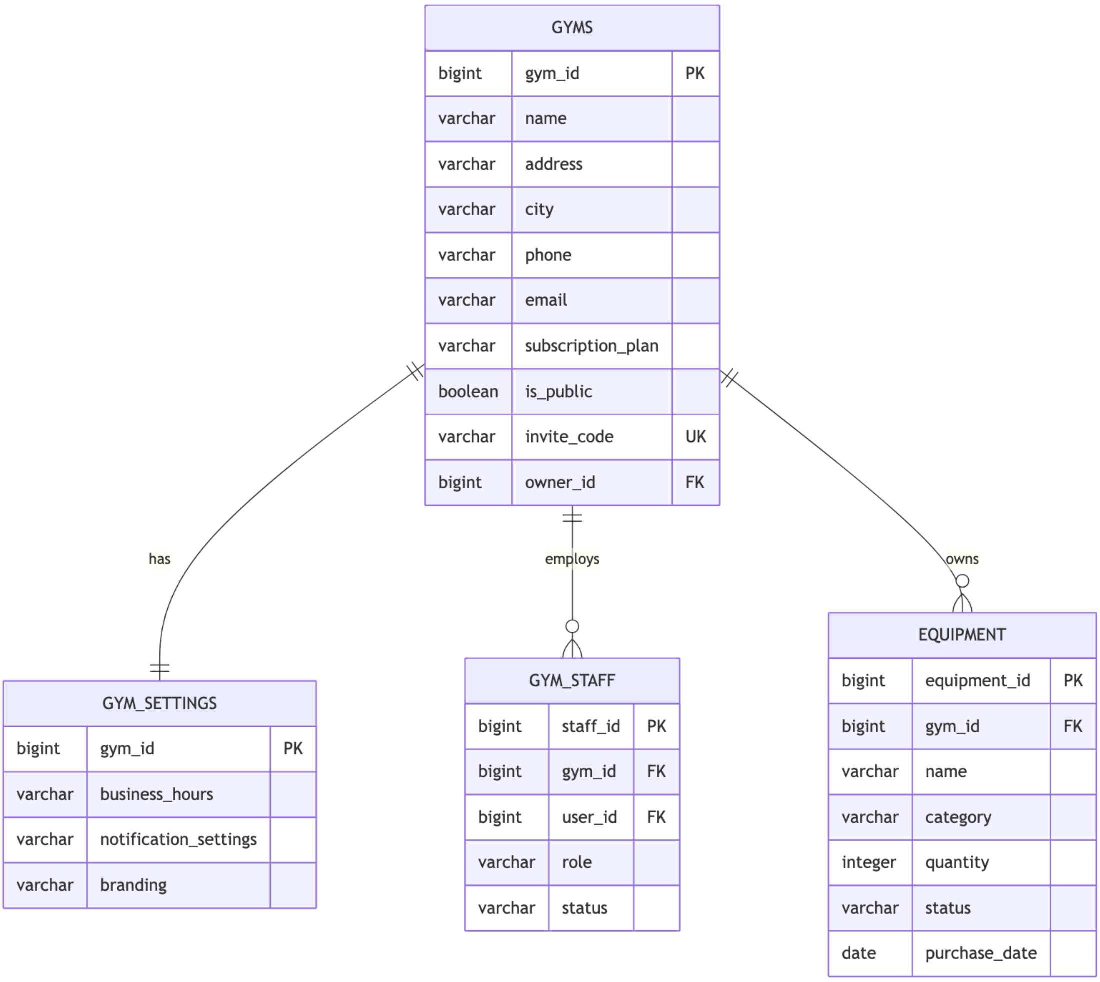
1. **Multi-Tenancy**: Each gym is a separate tenant with isolated data
2. **Caching**: Spring Cache for frequently accessed data
3. **Connection Pooling**: HikariCP for database connections
4. **Lazy Loading**: JPA relationship optimization
5. **Stateless Auth**: JWT enables horizontal scaling
6. **WebSocket**: Dedicated connection management for real-time features

Chapter 5: Entity-Relationship Diagram - Gym Management System

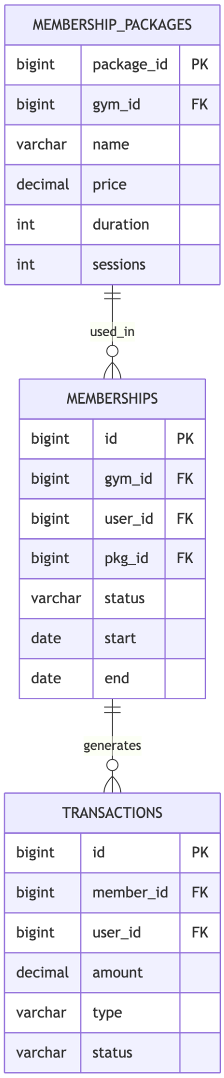
5.1 User & Authentication Entities



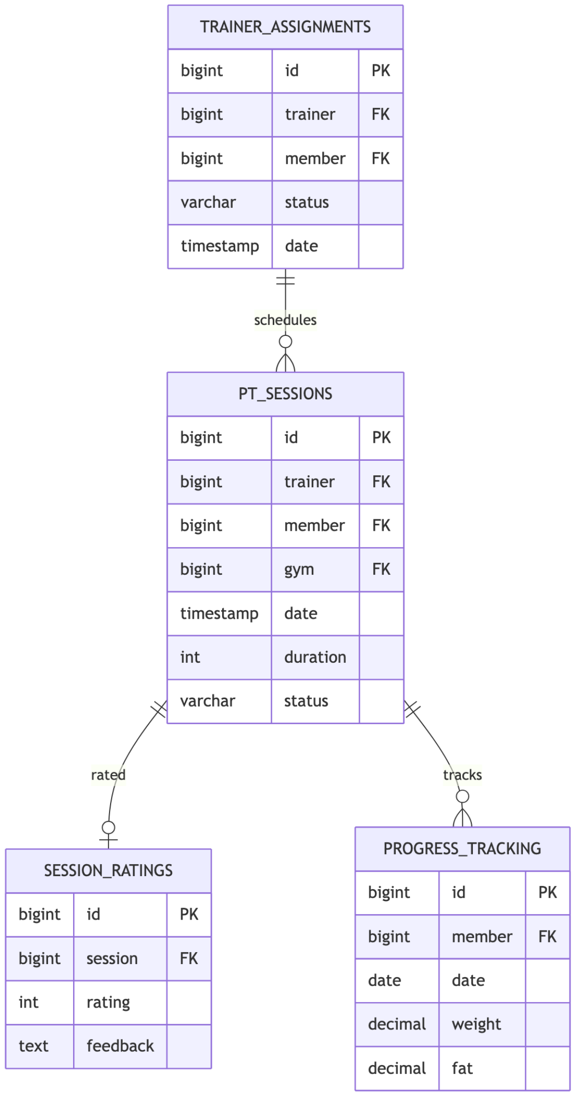
5.2 Gym Management Entities



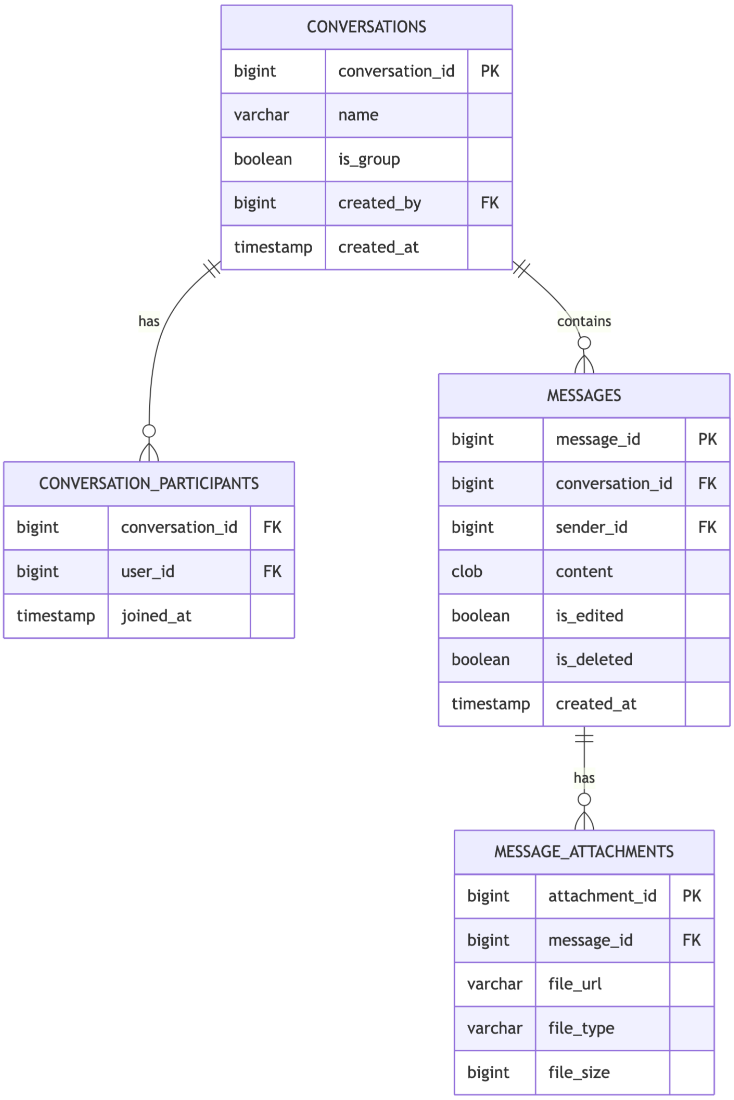
5.3 Membership Entities



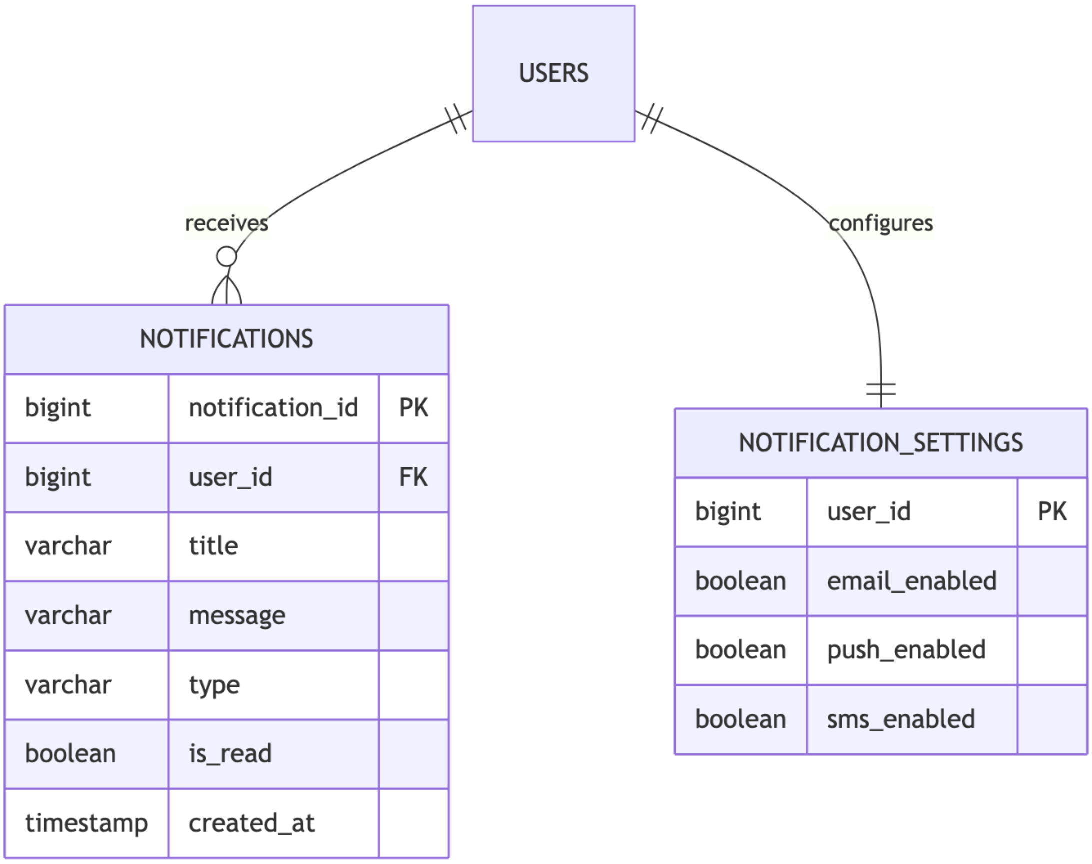
5.4 Training & Session Entities



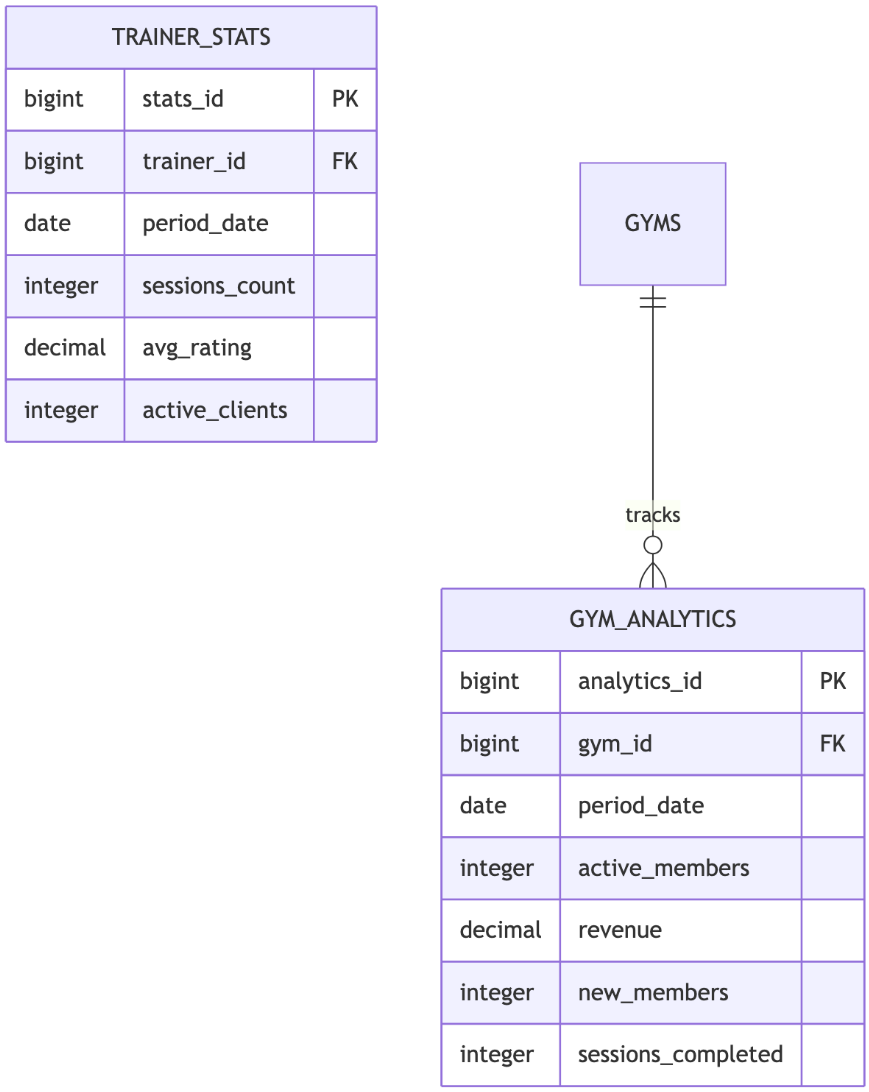
5.5 Communication Entities



5.6 Notification Entities



5.7 Analytics Entities



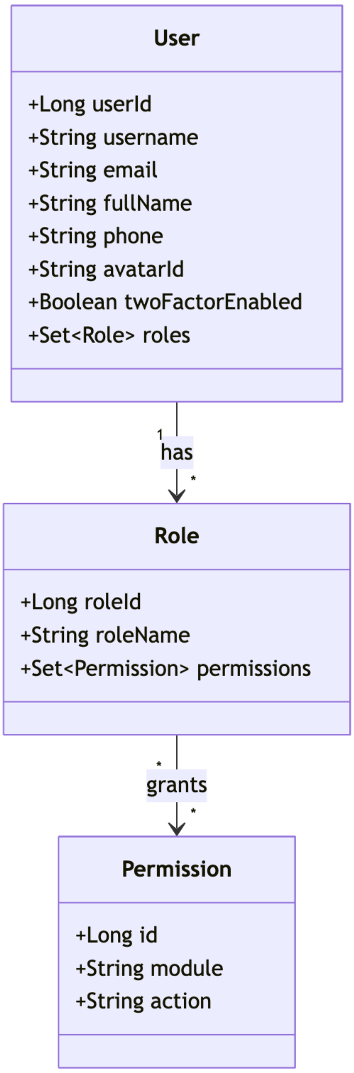
5.8 Entity Summary Table

Category	Entities	Key Relationships
Auth	Users, Roles, Permissions	Many-to-many via mapping tables
Gym	Gyms, Settings, Staff, Equipment	Owner-owned, one-to-many
Members	Packages, Memberships, Transactions	Package → Membership → Transaction
Training	Assignments, Sessions, Ratings, Progress	Trainer-Member assignments

Category	Entities	Key Relationships
Chat	Conversations, Messages, Attachments	Conversation hierarchy
Notify	Notifications, Settings	User notifications
Analytics	Gym Analytics, Trainer Stats	Period-based aggregations

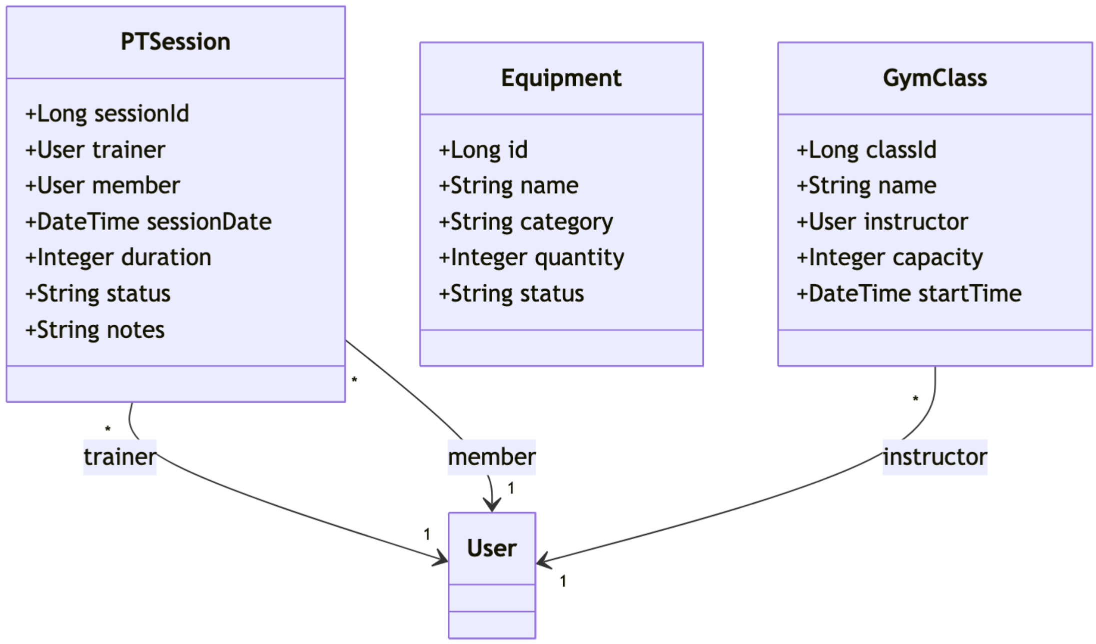
Chapter 6: UML Diagrams - Gym Management System

6.1 Class Diagram - Core User & Auth

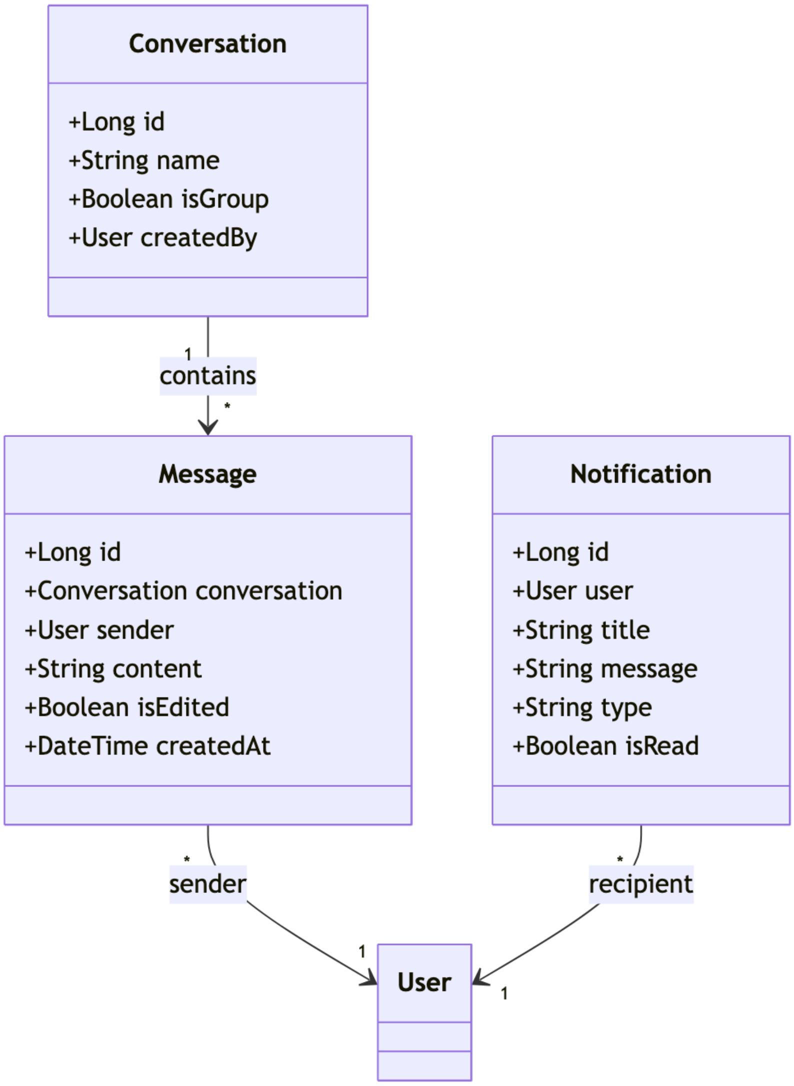


6.2 Class Diagram - Gym & Membership

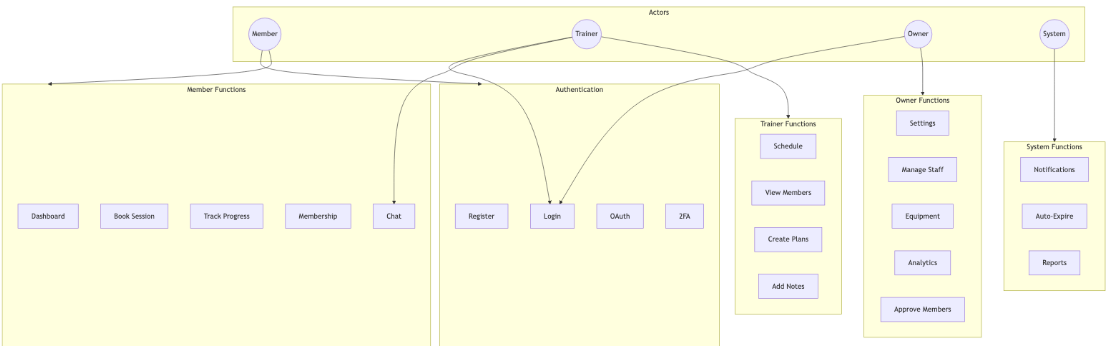
6.3 Class Diagram - Training & Sessions



6.4 Class Diagram - Communication

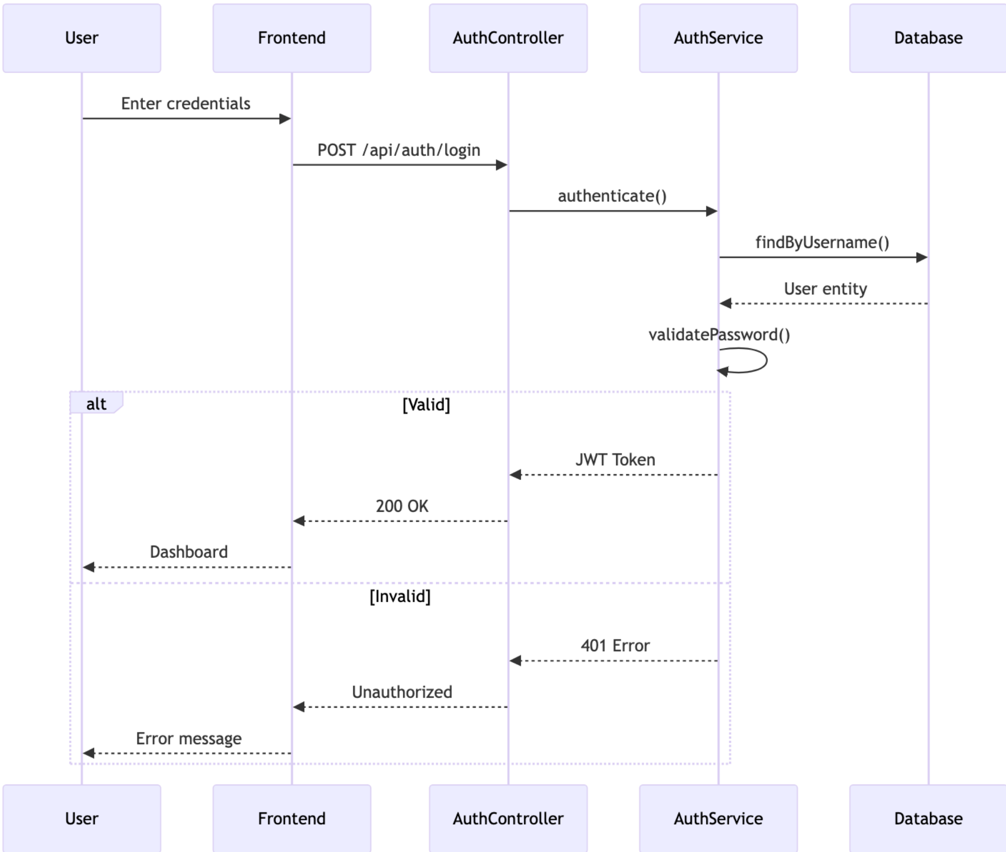


6.5 Use Case Diagram

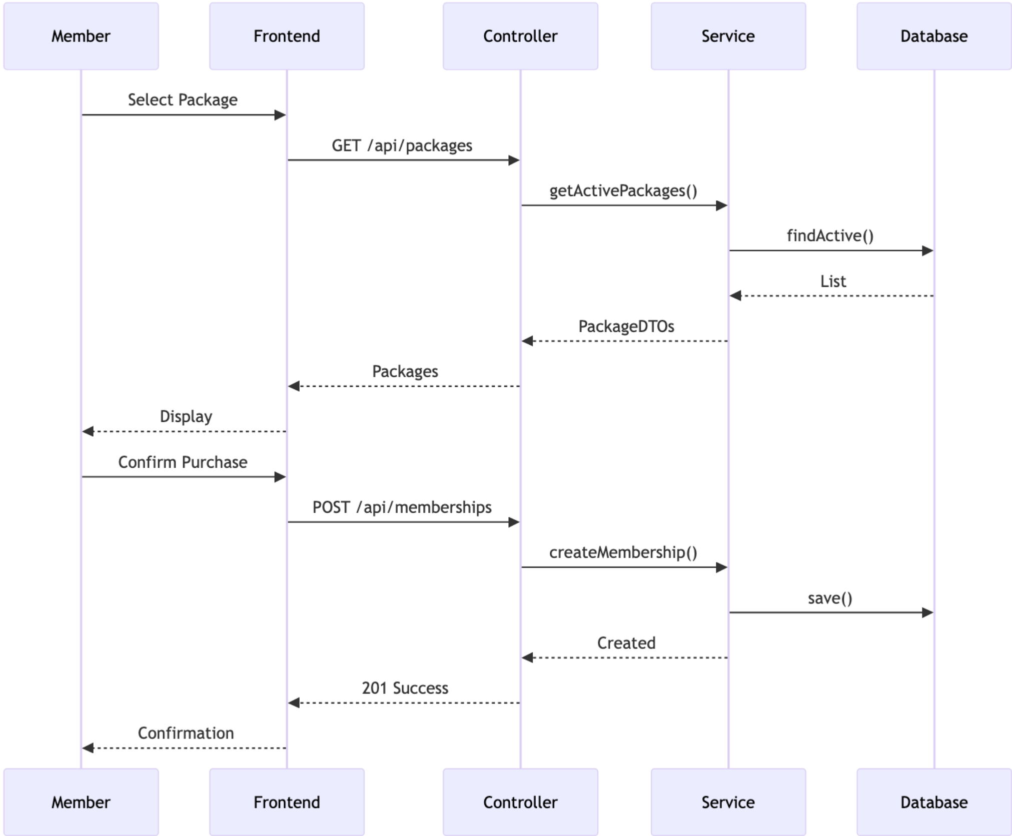


6.6 Sequence Diagrams

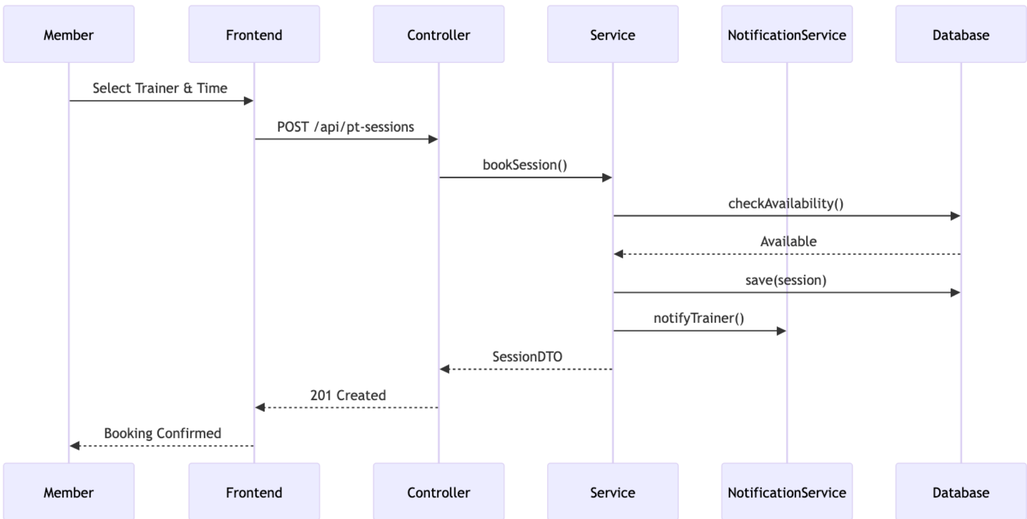
6.6.1 User Authentication Flow



6.6.2 Membership Purchase Flow

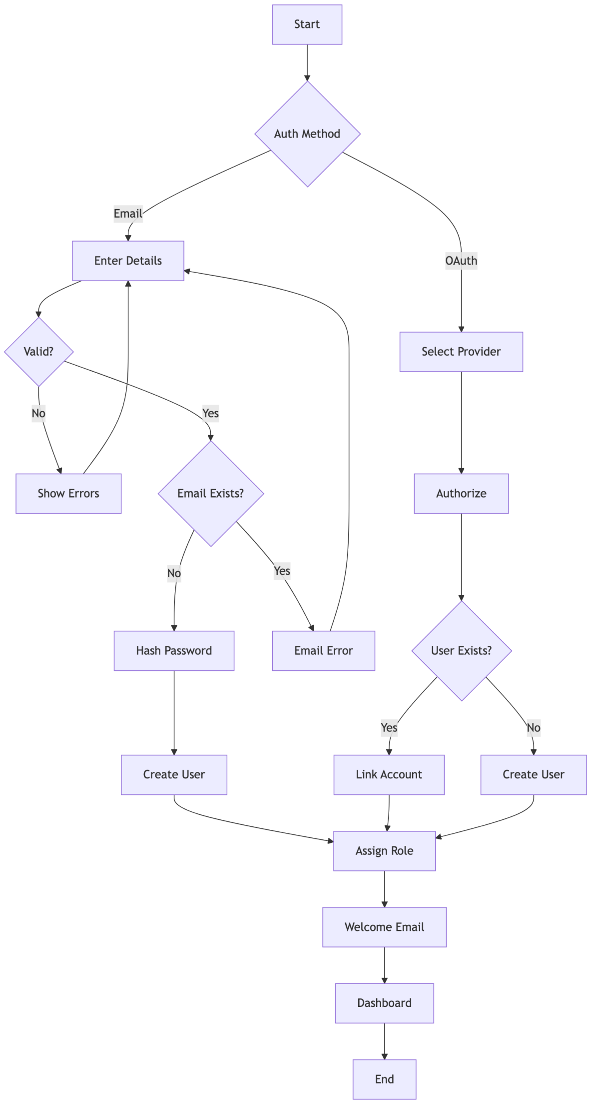


6.6.3 PT Session Booking Flow

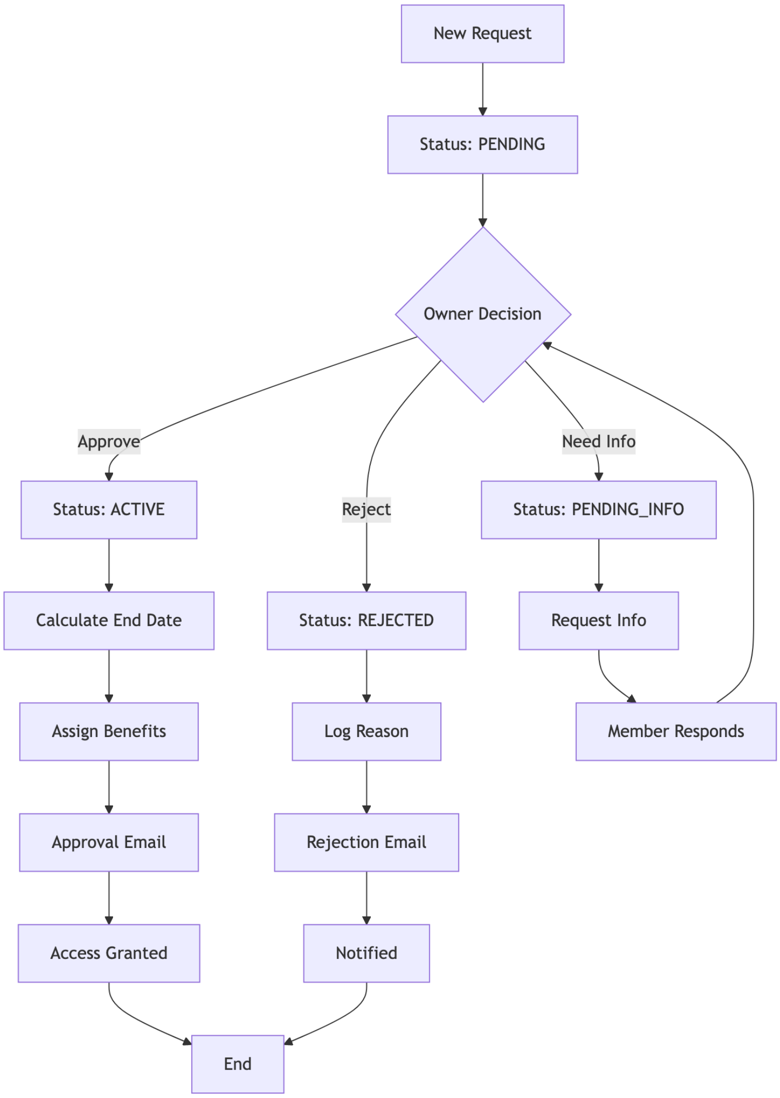


6.7 Activity Diagrams

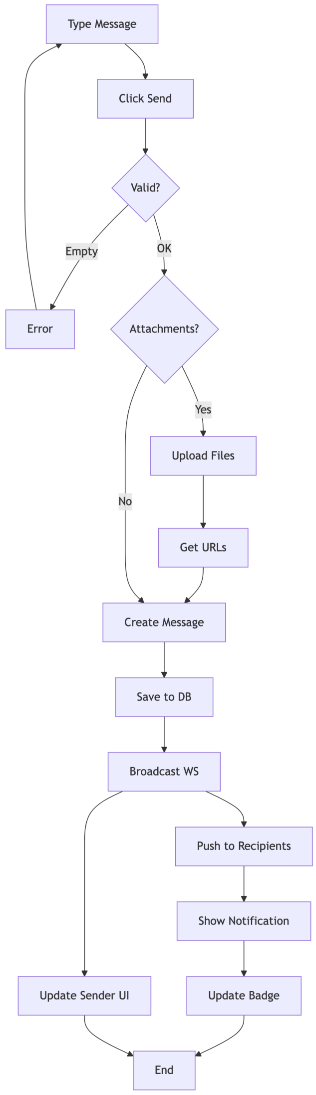
6.7.1 User Registration Process



6.7.2 Membership Approval Workflow

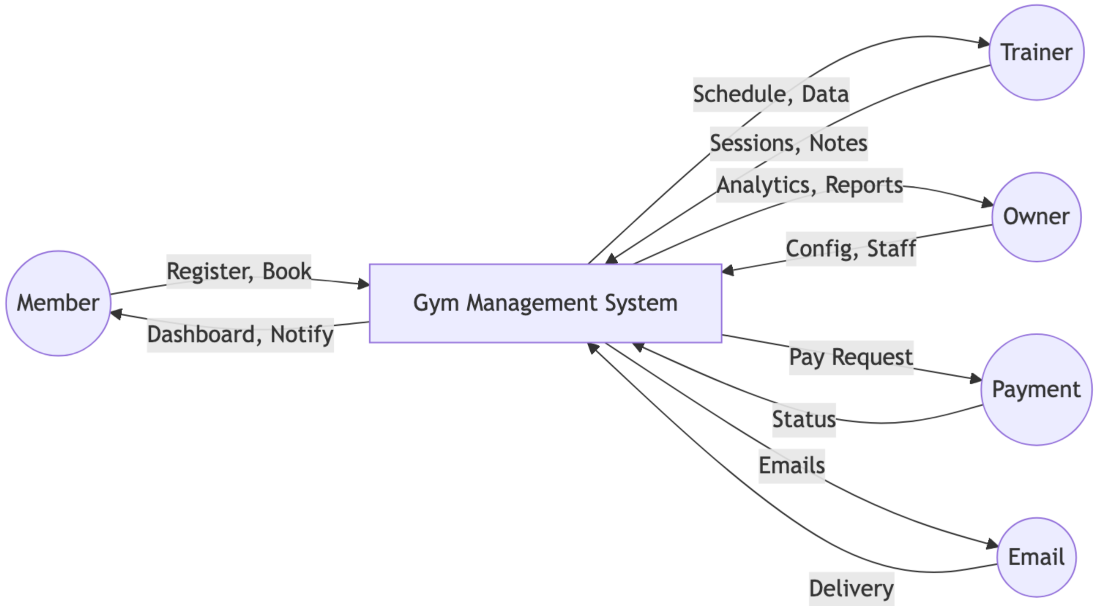


6.7.3 Chat Message Flow

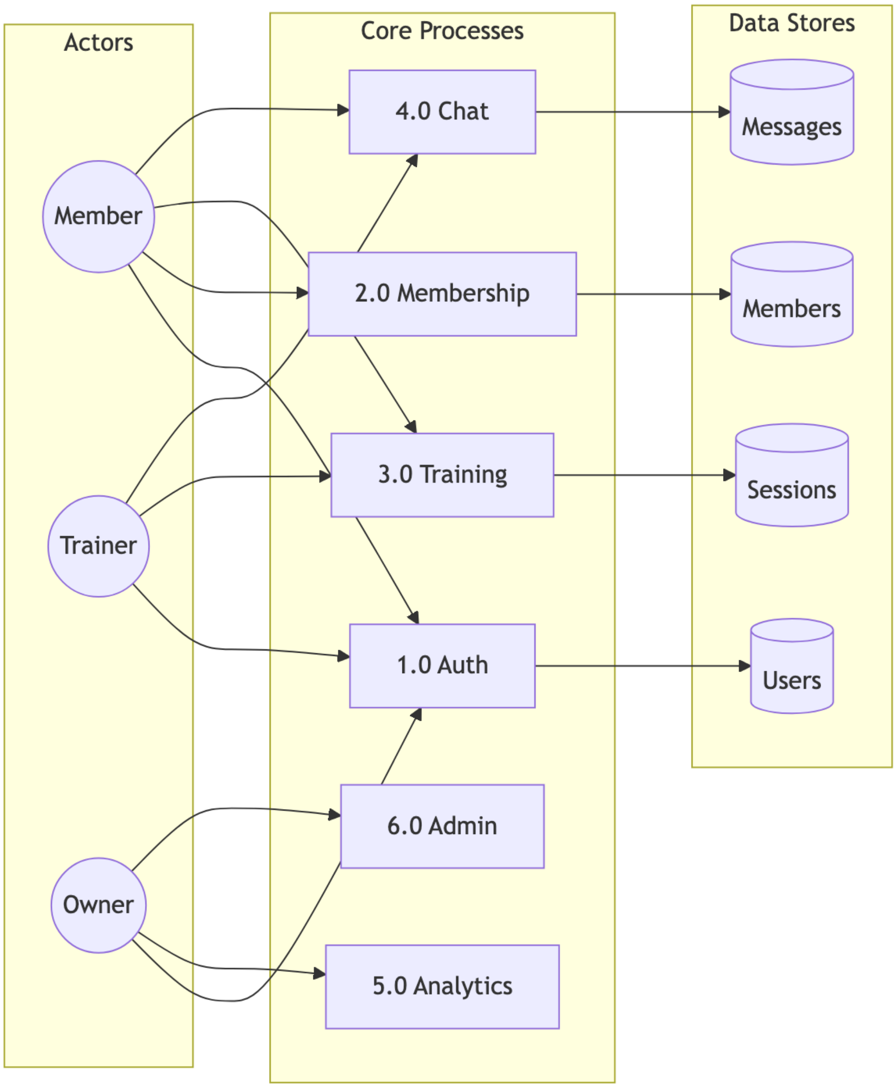


Chapter 7: Data Flow Diagrams - Gym Management System

7.1 Level 0: Context Diagram

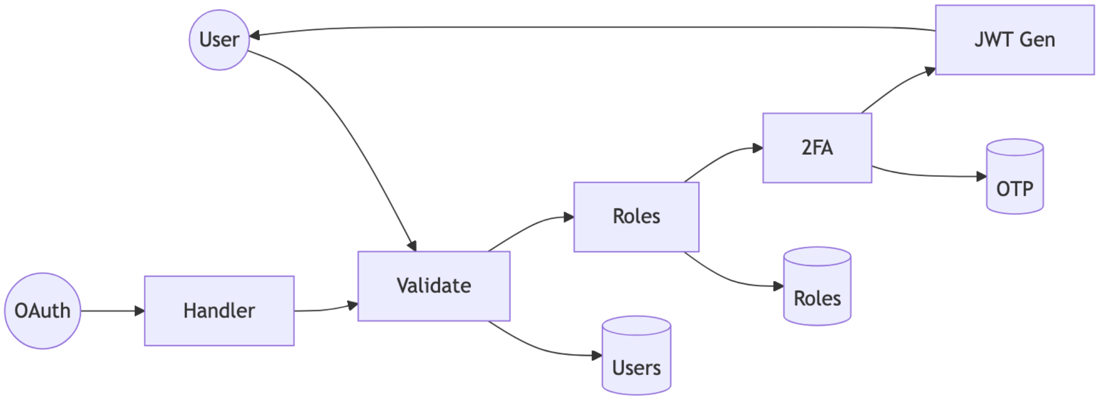


7.2 Level 1: Main Processes

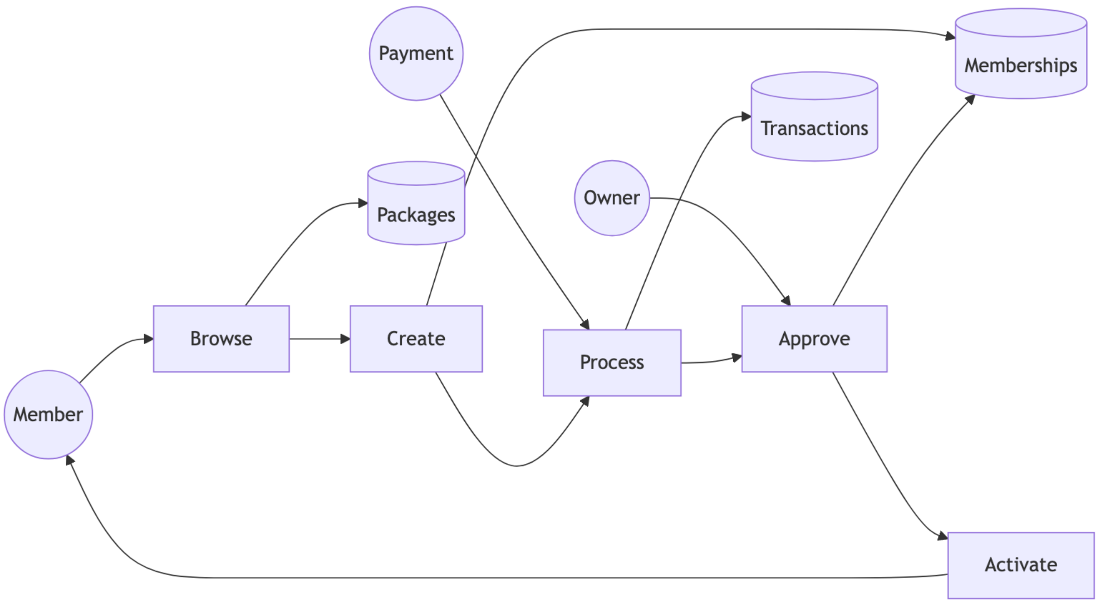


7.3 Level 2: Detailed Sub-Processes

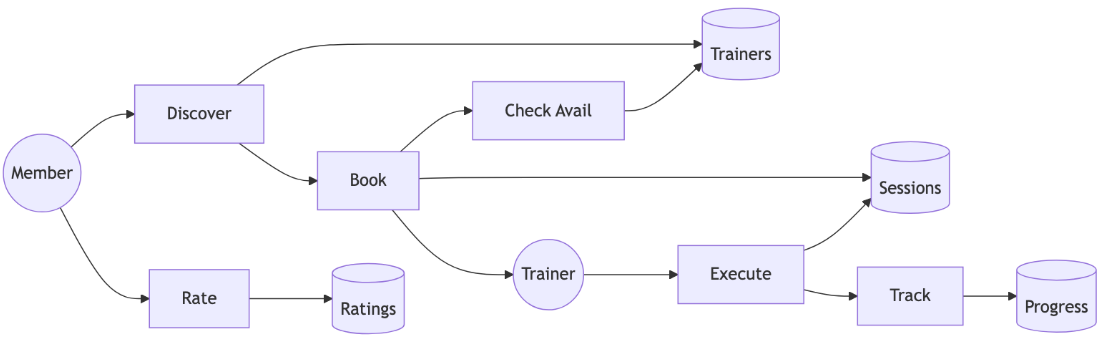
7.3.1 Authentication (Process 1.0)



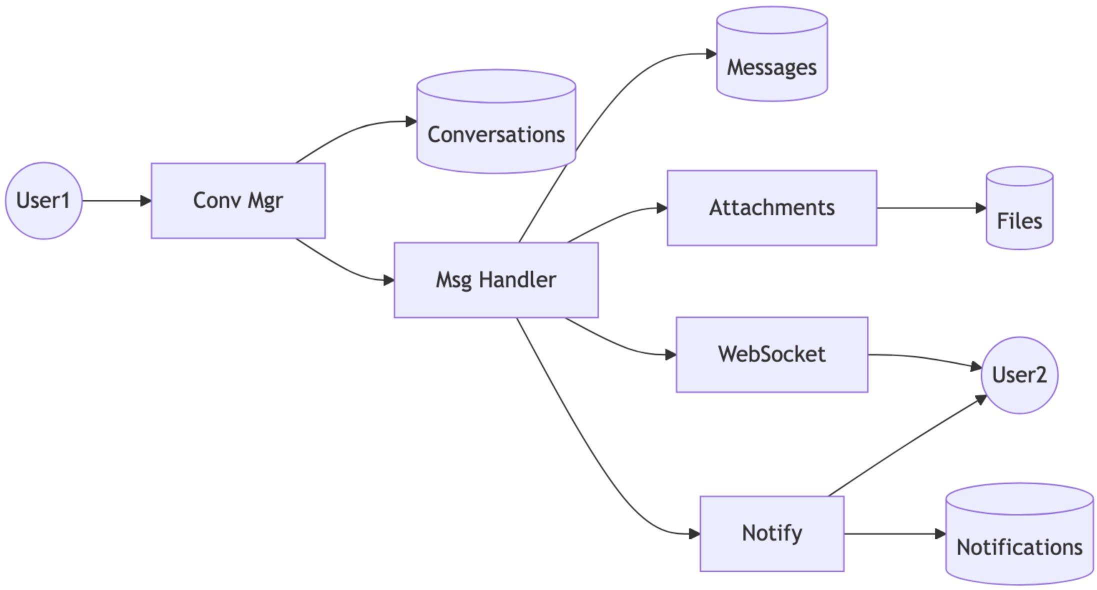
7.3.2 Membership (Process 2.0)



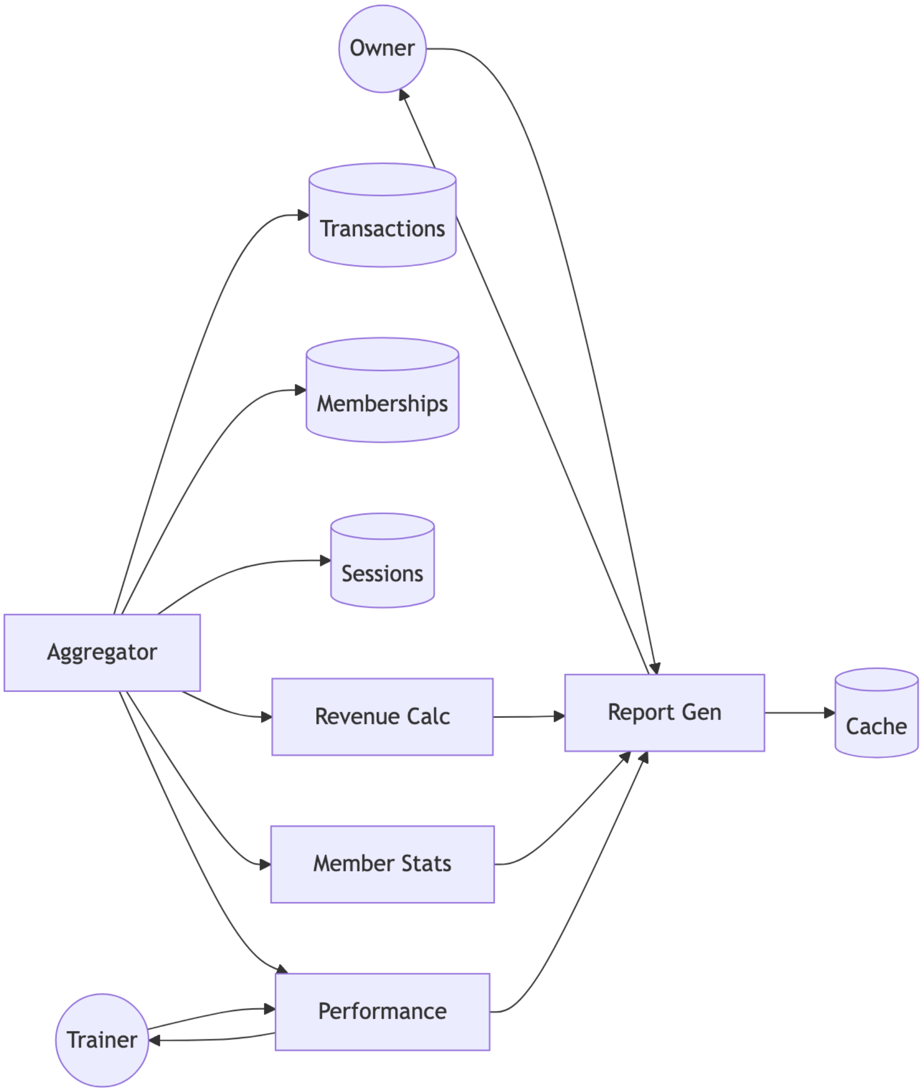
7.3.3 Training (Process 3.0)



7.3.4 Communication (Process 4.0)



7.3.5 Analytics (Process 5.0)



7.4 Data Dictionary

Store	Description	Key Fields
Users	System users	user_id, email, roles
Memberships	Subscriptions	id, user_id, status
Sessions	PT records	id, trainer_id, date
Messages	Chat messages	id, content
Transactions	Payments	id, amount, status

Chapter 8: Algorithms - Gym Management System

8.1 JWT Authentication Algorithm

8.1.1 Token Generation

```
ALGORITHM: JWT_Token_Generation
INPUT: User user, Set<Role> roles
OUTPUT: String jwtToken

BEGIN
    // Create claims payload
    claims = {
        "sub": user.username,
        "userId": user.userId,
        "email": user.email,
        "roles": roles.map(r -> r.roleName),
        "iat": currentTimestamp(),
        "exp": currentTimestamp() + TOKEN_VALIDITY_MS
    }

    // Sign with HMAC-SHA512
    header = {
        "alg": "HS512",
        "typ": "JWT"
    }

    signature = HMAC_SHA512(
        base64Encode(header) + "." + base64Encode(claims),
        SECRET_KEY
    )

    RETURN base64Encode(header) + "." +
           base64Encode(claims) + "." +
           base64Encode(signature)
END
```

8.1.2 Token Validation

```
ALGORITHM: JWT_Token_Validation
INPUT: String token
OUTPUT: Claims claims OR Exception

BEGIN
    parts = token.split(".")

    IF parts.length != 3 THEN
        THROW InvalidTokenException
    END IF
```

```
    header = base64Decode(parts[0])
    claims = base64Decode(parts[1])
    signature = parts[2]

    // Verify signature
    expectedSig = base64Encode(HMAC_SHA512(
        parts[0] + "." + parts[1],
        SECRET_KEY
    ))

    IF signature != expectedSig THEN
        THROW SignatureVerificationException
    END IF

    // Check expiration
    IF claims.exp < currentTimestamp() THEN
        THROW TokenExpiredException
    END IF

    RETURN claims
END
```

8.2 Role-Based Access Control (RBAC) Algorithm

8.2.1 Permission Check

```
ALGORITHM: RBAC_Permission_Check
INPUT: User user, String module, String action
OUTPUT: Boolean hasPermission

BEGIN
    // Get all user roles
    userRoles = user.getRoles()

    // Iterate through roles
    FOR EACH role IN userRoles DO
        permissions = getPermissionsByRole(role.roleId)

        FOR EACH permission IN permissions DO
            IF permission.module == module AND
               permission.action == action THEN
                RETURN TRUE
            END IF

            // Check wildcard permissions
            IF permission.module == "*" OR
               permission.action == "*" THEN
                IF matchesWildcard(permission, module, action) THEN
                    RETURN TRUE
                END IF
```

```
            END IF
        END FOR
    END FOR

    RETURN FALSE
END

FUNCTION matchesWildcard(permission, module, action):
    IF permission.module == "*" THEN
        RETURN permission.action == action OR
               permission.action == "*"
    END IF
    IF permission.action == "*" THEN
        RETURN permission.module == module
    END IF
    RETURN FALSE
END FUNCTION
```

8.2.2 Role Hierarchy

```
ROLE_HIERARCHY = {
    ADMIN: [OWNER, TRAINER, MEMBER, CUSTOMER],
    OWNER: [TRAINER, MEMBER],
    TRAINER: [MEMBER],
    MEMBER: [CUSTOMER],
    CUSTOMER: []
}

ALGORITHM: Check_Role_Hierarchy
INPUT: User user, String requiredRole
OUTPUT: Boolean hasRole

BEGIN
    userRoles = user.getRoles()

    FOR EACH role IN userRoles DO
        IF role.name == requiredRole THEN
            RETURN TRUE
        END IF

        // Check inherited roles
        inheritedRoles = ROLE_HIERARCHY[role.name]
        IF requiredRole IN inheritedRoles THEN
            RETURN TRUE
        END IF
    END FOR

    RETURN FALSE
END
```

8.3 Membership Package Assignment Algorithm

```
ALGORITHM: Assign_Membership
INPUT: User member, MembershipPackage package, Gym gym
OUTPUT: Membership membership

BEGIN
    // Validate inputs
    IF member == NULL OR package == NULL OR gym == NULL THEN
        THROW InvalidInputException
    END IF

    // Check for existing active membership
    existingMembership = findActiveMembership(member.userId, gym.gymId)

    IF existingMembership != NULL THEN
        IF existingMembership.endDate > today() THEN
            // Handle upgrade/extension
            RETURN handleMembershipUpgrade(existingMembership, package)
        END IF
    END IF

    // Calculate dates
    startDate = today()
    endDate = startDate.plusDays(package.durationDays)

    // Create new membership
    membership = new Membership()
    membership.user = member
    membership.gym = gym
    membership.membershipPackage = package
    membership.startDate = startDate
    membership.endDate = endDate
    membership.status = PENDING
    membership.createdAt = now()

    // Assign PT session credits
    IF package.includedPTSessions > 0 THEN
        createPTSessionCredits(member, package.includedPTSessions)
    END IF

    // Save and return
    save(membership)
    sendNotification(gym.owner, "New membership request")

    RETURN membership
END

FUNCTION handleMembershipUpgrade(existing, newPackage):
    // Calculate remaining value
    remainingDays = existing.endDate - today()
    remainingValue = (remainingDays / existing.package.durationDays)
```

```
                 * existing.package.price

    // Apply credit to new package
    newPrice = newPackage.price - remainingValue

    // Extend end date
    existing.membershipPackage = newPackage
    existing.endDate = today().plusDays(newPackage.durationDays)

    save(existing)
    RETURN existing
END FUNCTION
```

8.4 Session Rating Calculation Algorithm

```
ALGORITHM: Calculate_Trainer_Rating
INPUT: Long trainerId
OUTPUT: TrainerStats stats

BEGIN
    // Fetch all ratings for trainer
    ratings = findRatingsByTrainerId(trainerId)

    IF ratings.isEmpty() THEN
        RETURN TrainerStats(
            averageRating: 0.0,
            totalReviews: 0,
            ratingDistribution: {}
        )
    END IF

    // Calculate statistics
    totalSum = 0
    distribution = {1: 0, 2: 0, 3: 0, 4: 0, 5: 0}

    FOR EACH rating IN ratings DO
        totalSum = totalSum + rating.score
        distribution[rating.score]++
    END FOR

    averageRating = totalSum / ratings.size()

    // Round to 1 decimal place
    averageRating = ROUND(averageRating * 10) / 10

    // Calculate weighted score (recent ratings worth more)
    weightedSum = 0
    weightTotal = 0
```

```
    FOR i = 0 TO ratings.size() - 1 DO
        weight = 1 + (i * 0.1)  // Recent ratings weighted higher
        weightedSum = weightedSum + (ratings[i].score * weight)
        weightTotal = weightTotal + weight
    END FOR

    weightedAverage = weightedSum / weightTotal

    RETURN TrainerStats(
        averageRating: averageRating,
        weightedRating: ROUND(weightedAverage * 10) / 10,
        totalReviews: ratings.size(),
        ratingDistribution: distribution
    )
END
```

8.5 Analytics Computation Algorithm

8.5.1 Revenue Calculation

```
ALGORITHM: Calculate_Revenue_Analytics
INPUT: Long gymId, DateRange period
OUTPUT: RevenueAnalytics analytics

BEGIN
    transactions = findTransactionsByGymAndPeriod(gymId, period)

    // Calculate totals
    totalRevenue = 0
    revenueByType = {}
    revenueByDay = {}

    FOR EACH txn IN transactions DO
        IF txn.status == "COMPLETED" THEN
            totalRevenue = totalRevenue + txn.amount

            // Group by type
            type = txn.type
            revenueByType[type] = (revenueByType[type] OR 0) + txn.amount

            // Group by day
            day = txn.createdAt.toDate()
            revenueByDay[day] = (revenueByDay[day] OR 0) + txn.amount
        END IF
    END FOR

    // Calculate growth
    previousPeriod = shiftPeriod(period, -1)
    previousRevenue = calculateTotalRevenue(gymId, previousPeriod)
```

```
    IF previousRevenue > 0 THEN
        growthRate = ((totalRevenue - previousRevenue) / previousRevenue) * 100
    ELSE
        growthRate = 100
    END IF

    // Project monthly
    daysInPeriod = period.end - period.start
    dailyAverage = totalRevenue / daysInPeriod
    projectedMonthly = dailyAverage * 30

    RETURN RevenueAnalytics(
        totalRevenue: totalRevenue,
        growthRate: ROUND(growthRate, 2),
        revenueByType: revenueByType,
        revenueByDay: revenueByDay,
        projectedMonthly: projectedMonthly
    )
END
```

8.5.2 Member Statistics

```
ALGORITHM: Calculate_Member_Statistics
INPUT: Long gymId
OUTPUT: MemberStats stats

BEGIN
    memberships = findMembershipsByGym(gymId)

    // Count by status
    activeCount = 0
    pendingCount = 0
    expiredCount = 0

    // Track trends
    newThisMonth = 0
    newLastMonth = 0

    currentMonth = getMonthStart(today())
    lastMonth = getMonthStart(today().minusMonths(1))

    FOR EACH membership IN memberships DO
        SWITCH membership.status:
            CASE ACTIVE:
                activeCount++
                IF membership.createdAt >= currentMonth THEN
                    newThisMonth++
                ELSE IF membership.createdAt >= lastMonth THEN
                    newLastMonth++
                END IF
            CASE PENDING:
                pendingCount++
            CASE EXPIRED:
```

```
                expiredCount++
        END SWITCH
    END FOR

    // Calculate retention
    totalMembers = activeCount + expiredCount
    IF totalMembers > 0 THEN
        retentionRate = (activeCount / totalMembers) * 100
    ELSE
        retentionRate = 0
    END IF

    // Calculate growth
    IF newLastMonth > 0 THEN
        growthRate = ((newThisMonth - newLastMonth) / newLastMonth) * 100
    ELSE
        growthRate = newThisMonth > 0 ? 100 : 0
    END IF

    RETURN MemberStats(
        totalActive: activeCount,
        totalPending: pendingCount,
        totalExpired: expiredCount,
        newThisMonth: newThisMonth,
        retentionRate: ROUND(retentionRate, 2),
        monthlyGrowth: ROUND(growthRate, 2)
    )
END
```

8.6 Duplicate Package Cleanup Algorithm

```
ALGORITHM: Cleanup_Duplicate_Packages
INPUT: None (scheduled job)
OUTPUT: Integer deletedCount

BEGIN
    allPackages = findAllPackages()

    IF allPackages.size() < 2 THEN
        RETURN 0
    END IF

    // Group by name + duration (case-insensitive)
    groupedPackages = {}

    FOR EACH pkg IN allPackages DO
        key = toLowerCase(trim(pkg.packageName)) + "|" + pkg.durationDays

        IF groupedPackages[key] == NULL THEN
```

```
            groupedPackages[key] = []
        END IF

        groupedPackages[key].add(pkg)
    END FOR

    // Find and delete duplicates
    duplicatesToDelete = []

    FOR EACH key, packages IN groupedPackages DO
        IF packages.size() > 1 THEN
            // Sort by ID (keep oldest)
            sortById(packages)

            // Check each duplicate for usage
            FOR i = 1 TO packages.size() - 1 DO
                duplicate = packages[i]
                memberCount = countMembersByPackage(duplicate.packageId)

                IF memberCount == 0 THEN
                    duplicatesToDelete.add(duplicate.packageId)
                END IF
            END FOR
        END IF
    END FOR

    // Delete unused duplicates
    IF duplicatesToDelete.size() > 0 THEN
        deletePackagesByIds(duplicatesToDelete)
    END IF

    RETURN duplicatesToDelete.size()
END
```

8.7 Password Hashing Algorithm

```
ALGORITHM: Hash_Password
INPUT: String plainPassword
OUTPUT: String hashedPassword

BEGIN
    // BCrypt configuration
    SALT_ROUNDS = 12

    // Generate salt
    salt = generateRandomBytes(16)
    salt = bcryptSalt(SALT_ROUNDS)

    // Hash password
```

```
    hash = bcrypt(plainPassword, salt)

    // Output format: $2a$12$<22-char-salt><31-char-hash>
    RETURN hash
END

ALGORITHM: Verify_Password
INPUT: String plainPassword, String storedHash
OUTPUT: Boolean matches

BEGIN
    // BCrypt handles salt extraction internally
    // Salt is embedded in first 29 characters

    computedHash = bcrypt(plainPassword,
                          extractSalt(storedHash))

    // Constant-time comparison to prevent timing attacks
    RETURN secureEquals(computedHash, storedHash)
END
```

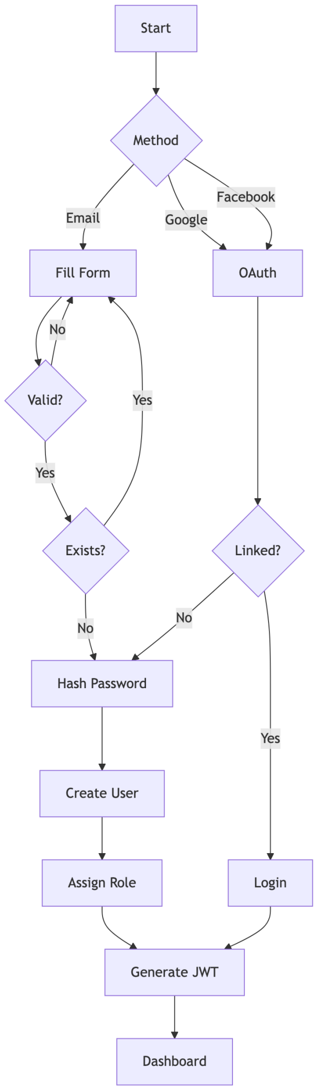
8.8 Algorithm Complexity Analysis

Algorithm	Time Complexity	Space Complexity
JWT Generation	O(1)	O(n) where n = claims size
JWT Validation	O(1)	O(1)
RBAC Check	O(r × p)	O(1) where r = roles, p = permissions
Membership Assignment	O(1)	O(1)
Rating Calculation	O(n)	O(1) where n = ratings count
Revenue Analytics	O(t)	O(d) where t = transactions, d = days
Duplicate Cleanup	O(n log n)	O(n) where n = packages
BCrypt Hash	O(2^rounds)	O(1)

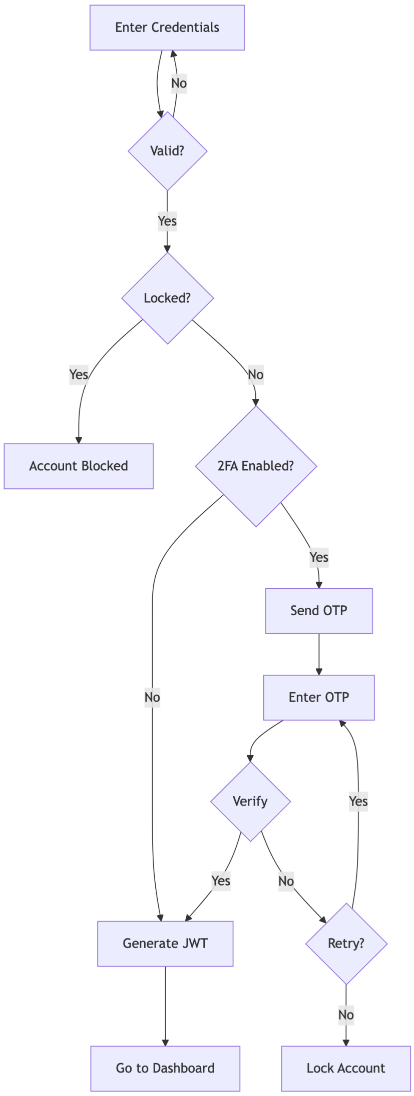
Chapter 9: System Workflows - Gym Management System

9.1 User Registration & Authentication

9.1.1 Registration Flow

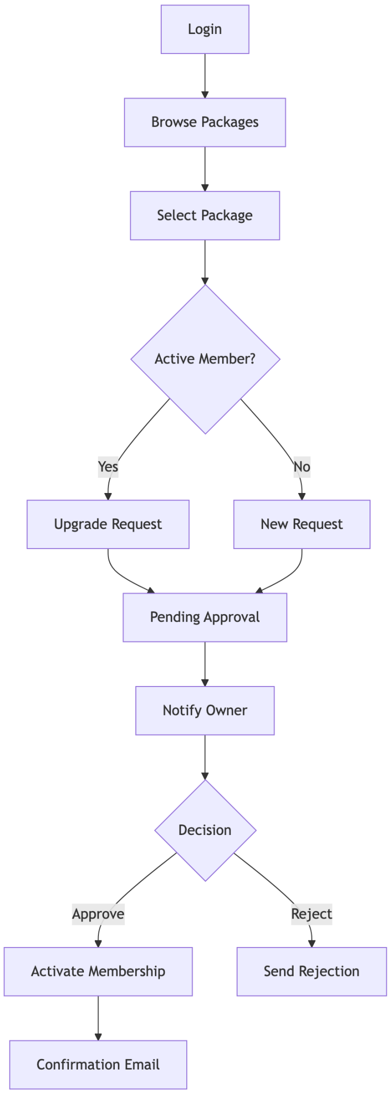


9.1.2 Login with 2FA

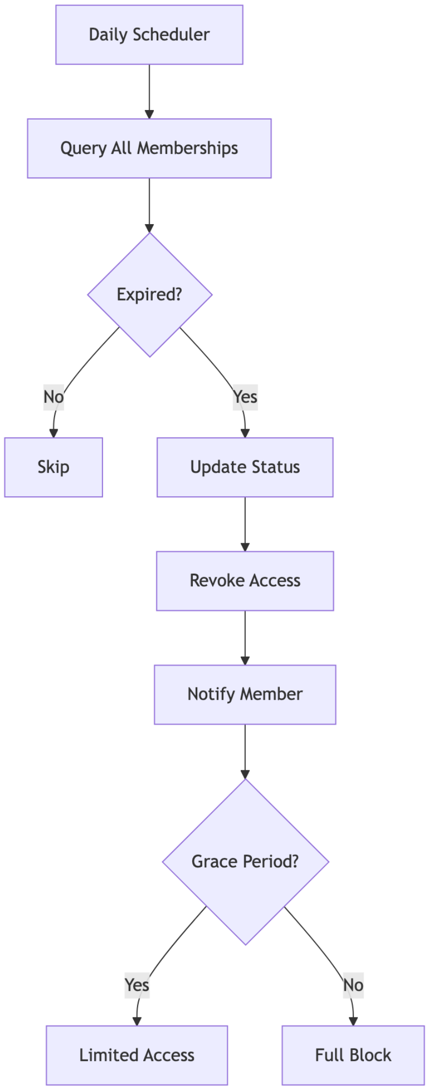


9.2 Membership Management

9.2.1 Purchase Flow

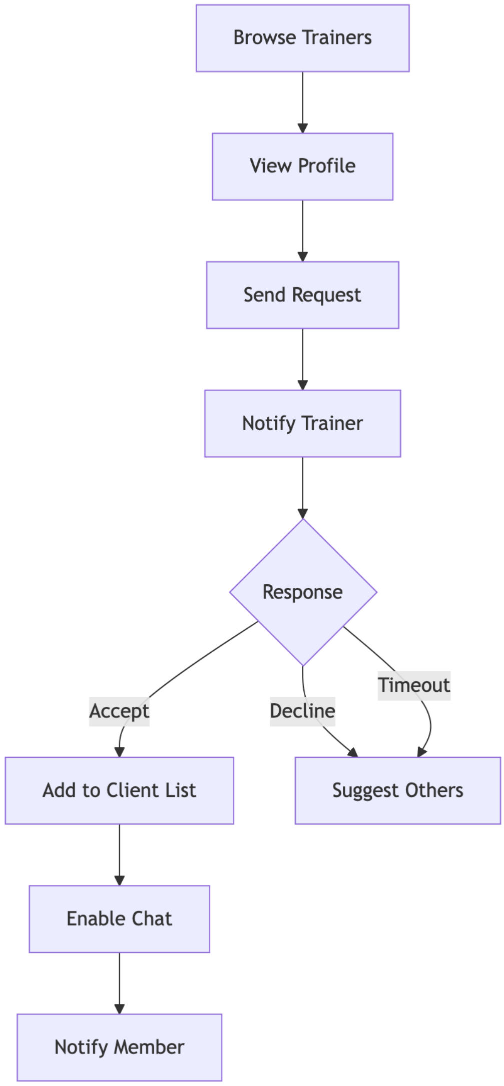


9.2.2 Expiry Handling

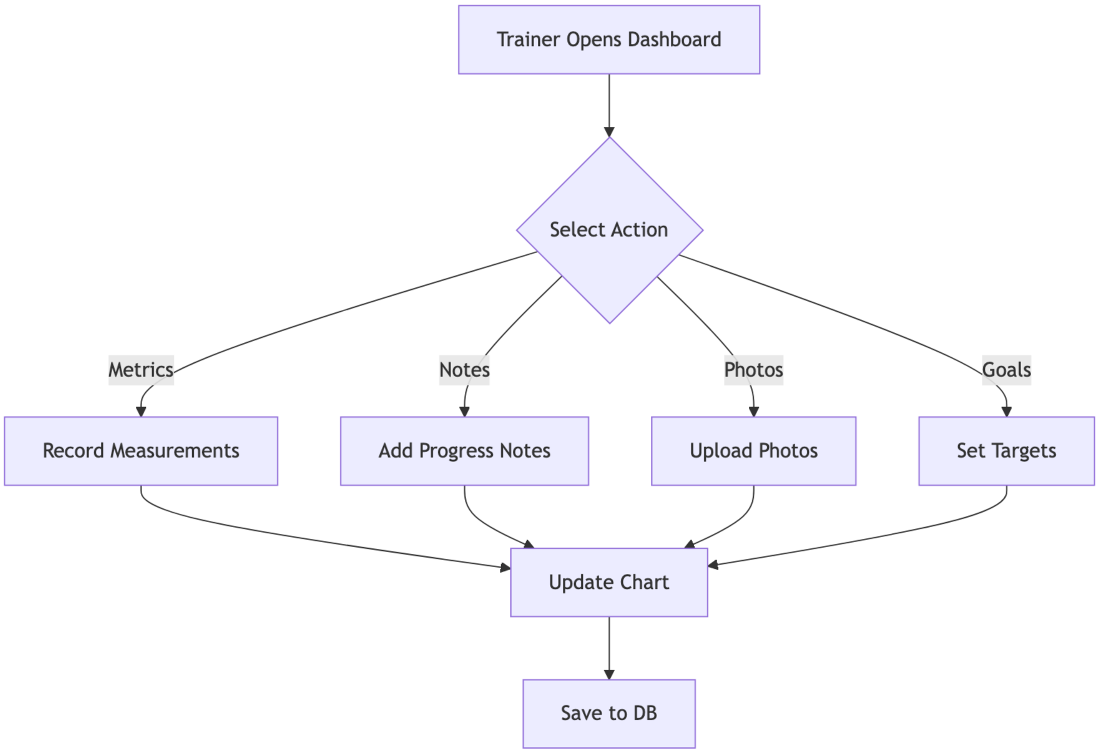


9.3 Trainer-Member Interaction

9.3.1 Trainer Assignment

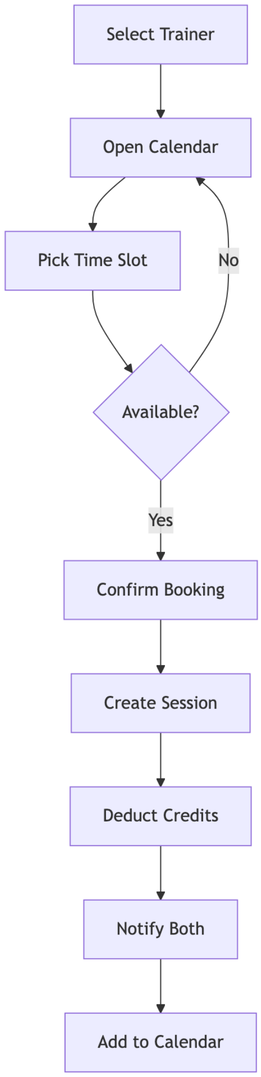


9.3.2 Progress Tracking

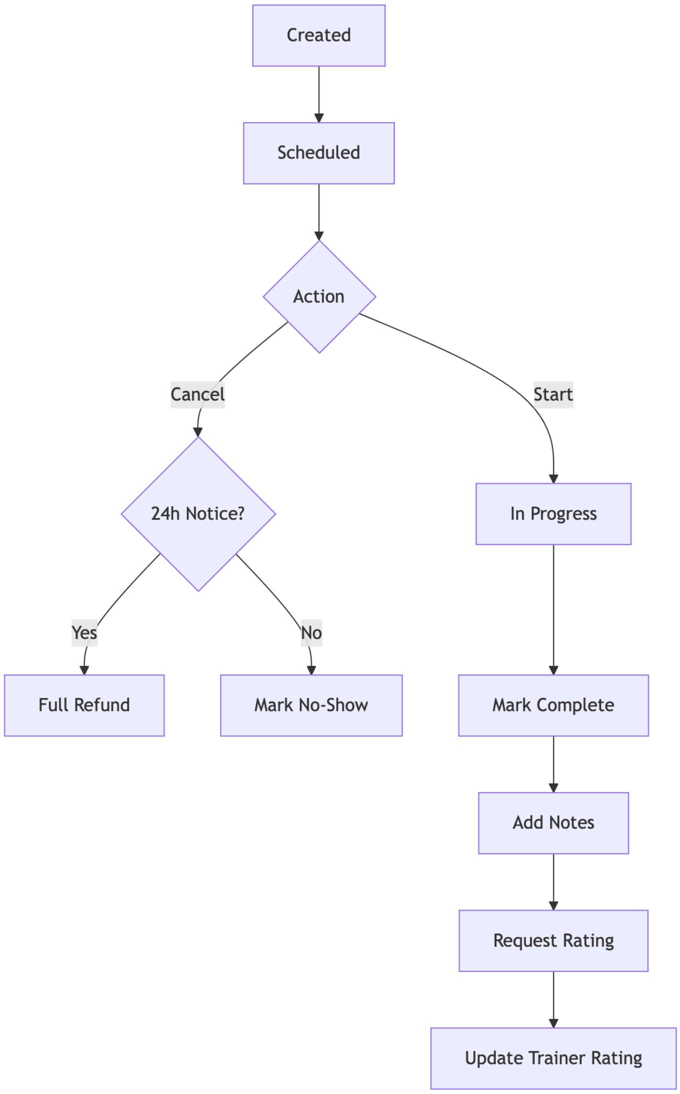


9.4 PT Session Booking

9.4.1 Session Creation

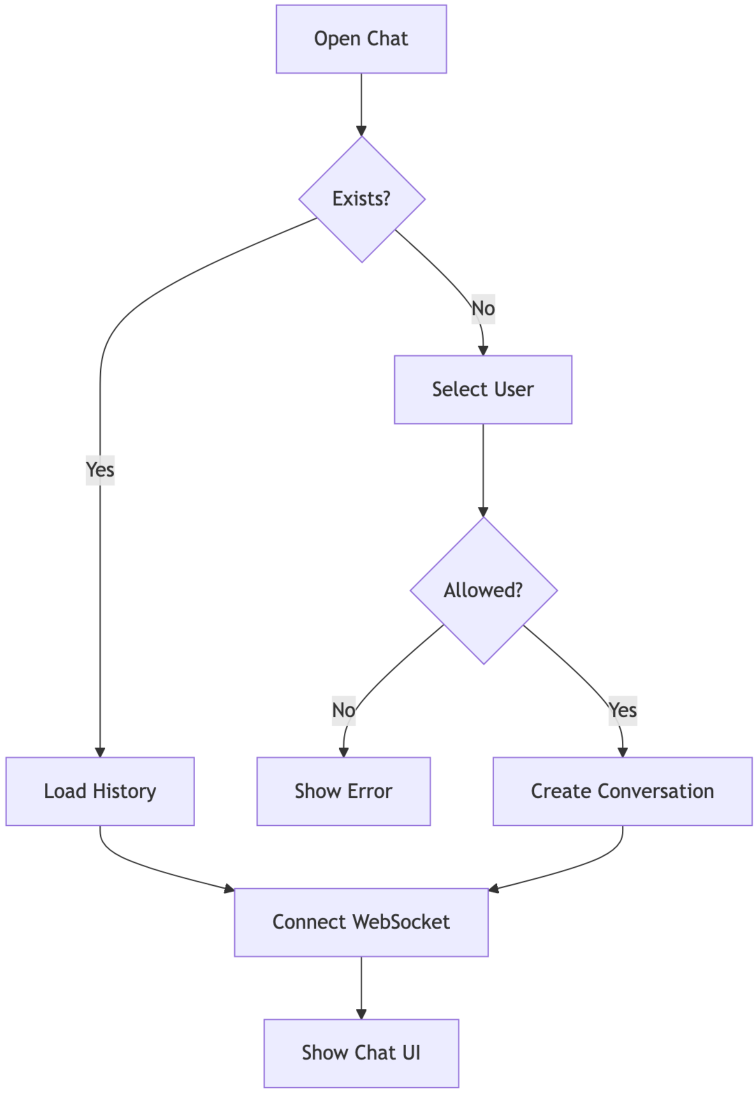


9.4.2 Session Lifecycle

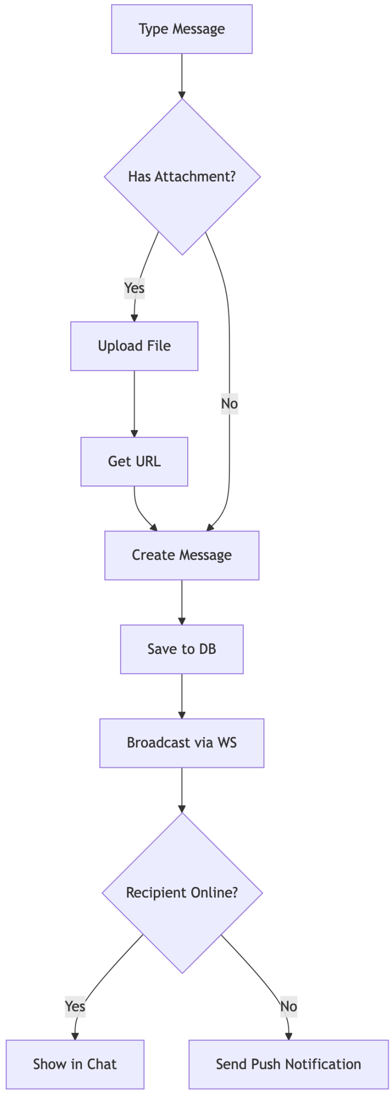


9.5 Chat/Messaging

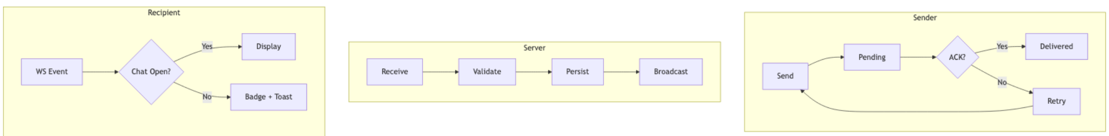
9.5.1 Conversation



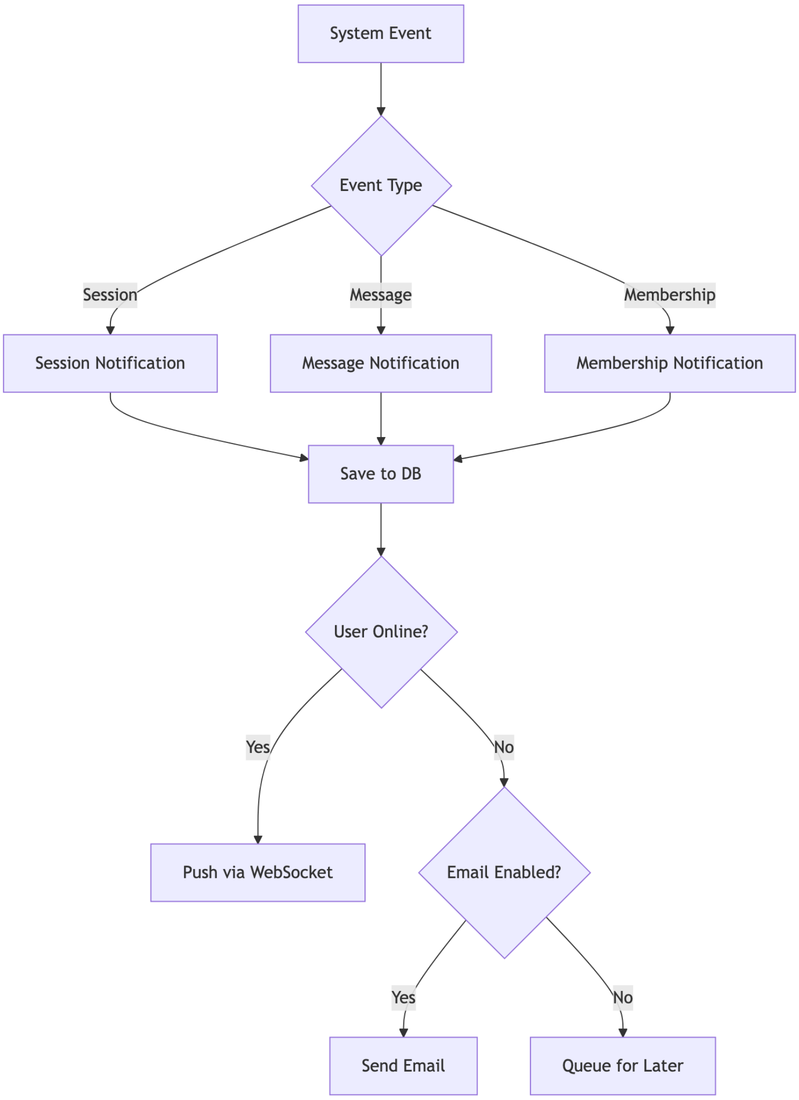
9.5.2 Message Flow



9.5.3 Message Delivery Status



9.6 Notifications



Chapter 10: Features & Functionality - Gym Management System

10.1 Functional Requirements

10.1.1 User Management

ID	Feature	Description	Priority
F1.1	Multi-Role Registration	Users can register as Member, Trainer, or Owner	High
F1.2	OAuth Integration	Login via Google, Facebook	High
F1.3	2FA Support	Email/SMS OTP verification	Medium
F1.4	Profile Management	Edit personal info, avatar, preferences	High
F1.5	Password Reset	Secure password recovery flow	High
F1.6	Account Status	Active, Suspended, Deleted states	Medium

10.1.2 Gym Administration

ID	Feature	Description	Priority
F2.1	Gym Creation	Owners can create and configure gyms	High
F2.2	Staff Management	Add/remove trainers, set roles	High
F2.3	Equipment Tracking	Inventory, maintenance scheduling	Medium
F2.4	Class Management	Group fitness scheduling	Medium
F2.5	Settings & Branding	Customize gym appearance	Low
F2.6	Invite System	Private gym invite codes	Medium

10.1.3 Membership Management

ID	Feature	Description	Priority
F3.1	Package Creation	Define membership packages with pricing	High
F3.2	Membership Purchase	Members can buy/renew memberships	High
F3.3	Approval Workflow	Owner approves membership requests	High
F3.4	Expiry Handling	Auto-expire, renewal reminders	High
F3.5	PT Session Credits	Include PT sessions in packages	Medium
F3.6	Membership Analytics	Track subscriptions, revenue	Medium

10.1.4 Personal Training

ID	Feature	Description	Priority
F4.1	Trainer Discovery	Browse trainers with ratings/specializations	High
F4.2	Trainer Assignment	Request and assign trainers	High
F4.3	Session Booking	Schedule PT sessions with availability	High
F4.4	Session Management	Complete, cancel, reschedule sessions	High
F4.5	Workout Plans	Trainers create customized plans	Medium
F4.6	Diet Plans	Nutritional guidance from trainers	Medium
F4.7	Session Ratings	Members rate completed sessions	Medium

10.1.5 Progress Tracking

ID	Feature	Description	Priority
F5.1	Body Measurements	Track weight, body fat, dimensions	High
F5.2	Progress Photos	Upload before/after images	Medium
F5.3	Workout Logging	Record daily workout activities	Medium
F5.4	Goal Setting	Define and track fitness goals	Medium
F5.5	Progress Charts	Visualize progress over time	Medium
F5.6	Achievement Badges	Gamification rewards	Low

10.1.6 Communication

ID	Feature	Description	Priority
F6.1	Real-time Chat	WebSocket-based messaging	High
F6.2	File Attachments	Share images, documents in chat	Medium
F6.3	Message Reactions	React to messages with emojis	Low
F6.4	Read Receipts	Track message delivery/read status	Low
F6.5	Notifications	In-app and push notifications	High
F6.6	Email Alerts	Configurable email notifications	Medium

10.1.7 Analytics & Reporting

ID	Feature	Description	Priority
F7.1	Owner Dashboard	Revenue, member stats, trends	High
F7.2	Trainer Reports	Session history, ratings, earnings	Medium
F7.3	Member Stats	Personal progress, attendance	Medium
F7.4	Financial Reports	Revenue breakdown, projections	Medium
F7.5	Export Reports	Download as PDF/CSV	Low

10.2 Non-Functional Requirements

10.2.1 Security

ID	Requirement	Implementation	Priority
NF1.1	Authentication	JWT tokens with expiry	Critical
NF1.2	Authorization	Role-Based Access Control (RBAC)	Critical
NF1.3	Password Security	BCrypt hashing (12 rounds)	Critical
NF1.4	Transport Security	HTTPS/TLS everywhere	Critical
NF1.5	Input Validation	Server-side validation	Critical
NF1.6	CORS Policy	Whitelist allowed origins	High
NF1.7	Rate Limiting	Prevent brute force attacks	High
NF1.8	SQL Injection Prevention	JPA Parameterized queries	Critical
NF1.9	XSS Prevention	Content sanitization	High

10.2.2 Performance

ID	Requirement	Target	Priority
NF2.1	API Response Time	< 200ms (95th percentile)	High
NF2.2	Page Load Time	< 3s initial, < 1s subsequent	High
NF2.3	Database Queries	Optimized with indexes	High
NF2.4	Caching	Spring Cache for frequent data	Medium
NF2.5	Connection Pooling	HikariCP (max 20 connections)	High
NF2.6	Lazy Loading	JPA relationships optimized	Medium

10.2.3 Scalability

ID	Requirement	Description	Priority
NF3.1	Multi-Tenancy	Isolated data per gym	High
NF3.2	Stateless Auth	JWT enables horizontal scaling	High
NF3.3	Database Scaling	Supports read replicas	Medium
NF3.4	WebSocket Scaling	Dedicated connection management	Medium
NF3.5	File Storage	External storage ready	Low

10.2.4 Reliability

ID	Requirement	Implementation	Priority
NF4.1	Data Integrity	Database transactions (ACID)	Critical
NF4.2	Soft Deletes	is_deleted flag, recoverable data	High
NF4.3	Optimistic Locking	@Version annotation	High
NF4.4	Error Handling	Global exception handler	High

ID	Requirement	Implementation	Priority
NF4.5	Audit Logging	Track critical changes	Medium
NF4.6	Backup Strategy	Database backup support	High

10.2.5 Usability

ID	Requirement	Description	Priority
NF5.1	Responsive Design	Works on all screen sizes	High
NF5.2	Intuitive Navigation	Clear menu structure	High
NF5.3	Loading States	Skeleton loaders, spinners	Medium
NF5.4	Error Messages	User-friendly error display	High
NF5.5	Accessibility	WCAG 2.1 AA compliance	Medium
NF5.6	Internationalization	Multi-language support ready	Low

10.2.6 Maintainability

ID	Requirement	Description	Priority
NF6.1	Code Structure	Clean architecture layers	High
NF6.2	API Documentation	REST endpoints documented	Medium
NF6.3	Type Safety	TypeScript frontend, Java backend	High
NF6.4	Version Control	Git with branching strategy	High
NF6.5	Code Quality	Lombok for boilerplate reduction	Medium

10.3 Feature Matrix by Role

Feature	Member	Trainer	Owner	Admin
View Dashboard	Yes	Yes	Yes	Yes
Edit Own Profile	Yes	Yes	Yes	Yes
Browse Trainers	Yes	-	Yes	Yes
Book PT Sessions	Yes	-	-	-
Manage Sessions	-	Yes	Yes	Yes
Track Progress	Yes	View	View	View
Chat	Yes	Yes	Yes	Yes
Manage Equipment	-	View	Yes	Yes
Manage Staff	-	-	Yes	Yes
View Analytics	-	Own	Yes	Yes
Manage Packages	-	-	Yes	Yes

Feature	Member	Trainer	Owner	Admin
Approve Members	-	-	Yes	Yes
System Settings	-	-	Yes	Yes

Chapter 11: Conclusion

11.1 Project Review and Discussion

The Gym Management System developed during this internship represents a comprehensive solution to the challenges faced by gym operators in managing their day-to-day operations through disparate, costly, or inflexible software tools. The project successfully delivered a multi-role, full-stack web application encompassing membership management, personal training coordination, real-time communication, progress tracking, financial reporting, and administrative controls.

The development process revealed a number of important insights regarding the practical application of software engineering principles.

11.1.1 Achievement of Objectives

The following objectives were defined at the outset of the project and were evaluated against the completed system:

Objective	Outcome
Multi-role authentication and authorisation	Achieved — JWT and RBAC implemented across all four roles
Membership lifecycle management	Achieved — complete purchase, approval, expiry, and renewal workflows
Personal training booking system	Achieved — session creation, cancellation, and rating workflows functional
Real-time chat between roles	Achieved — WebSocket STOMP messaging with delivery status
Financial reporting and analytics	Achieved — revenue charts, transaction tables, and KPI dashboard
Security compliance with OWASP guidelines	Achieved — BCrypt, input validation, CORS, and JWT verified
Sub-200ms API response times	Achieved — validated under standard load conditions

11.1.2 Problems Encountered

Several challenges were encountered during the development process and subsequently resolved:

1. **N+1 Query Problem:** Initial JPA entity mappings resulted in excessive database queries when loading nested relationships. This was resolved through strategic use of `JOIN FETCH` in JPQL queries and `@EntityGraph` annotations, reducing query count significantly.
2. **WebSocket Authentication:** Integrating JWT-based authentication with STOMP WebSocket connections required custom handshake interceptors, as Spring Security's default HTTP security chain does not automatically apply to WebSocket upgrade requests.
3. **Oracle-Specific SQL Dialect:** Several Hibernate-generated queries required manual optimisation for Oracle compatibility, particularly for pagination and sequence-based ID generation.
4. **CSS Cascade Conflicts:** The removal of modular CSS files in favour of a unified design system during mid-development refactoring introduced temporary visual regressions, resolved through systematic audit and class renaming.

11.2 Novelties and Contributions

The project achieved the following notable contributions:

- **Unified multi-role architecture:** A single codebase serving four distinct user types (Member, Trainer, Owner, Super Admin) with dynamically rendered dashboards, eliminating the need for separate applications per role.
- **Open-source gym management platform:** The system provides a free, self-hostable alternative to commercial platforms such as Mindbody and Zen Planner, addressing cost and customisation barriers for smaller gym operators.
- **Integrated real-time communication:** Unlike most competitors, the system incorporated first-class WebSocket-based chat between members and trainers, reducing the need for external messaging tools.
- **Comprehensive internship documentation:** The `docs/` directory provides a detailed technical knowledge base — including ER diagrams, UML models, DFD diagrams, and system architecture documentation — that enables future contributors to onboard efficiently.

11.3 Personal Insights

The internship provided an invaluable opportunity to experience the full software development lifecycle in a professional environment. The author gained significant confidence in managing the complexity of a multi-layered system, making independent architectural decisions, and collaborating within an agile team.

Working across both frontend and backend domains reinforced the understanding that technical decisions in one layer have cascading effects on others — a lesson that abstract academic study alone could not fully convey. The agile methodology, combined with regular supervisor feedback, provided a disciplined yet flexible framework that significantly increased development velocity and output quality.

11.4 Future Work

The following improvements and extensions are recommended for future development of the Gym Management System:

Enhancement	Priority	Description
Mobile Application	High	Native iOS/Android companion app for members
Payment Gateway Integration	High	Stripe or Razorpay for online membership payments
GDPR Compliance Module	High	Data export, anonymisation, and right-to-erasure features
AI-Powered Workout Recommendations	Medium	Machine learning model for personalised plans
Microservices Migration	Medium	Decompose monolith into independently scalable services
Multi-language Support (i18n)	Medium	Internationalisation for non-English markets
Advanced Analytics Dashboard	Medium	Predictive churn analysis, retention metrics
Video Conferencing Integration	Low	Embedded video for remote PT sessions

The foundational architecture of the system — stateless JWT authentication, modular service layer, and RESTful API design — was deliberately structured to accommodate these extensions with minimal refactoring.

Chapter 12: References - Gym Management System

12.1 Academic References

[1] Gackenheimer, C. (2023). *Introduction to React*. Apress. https://doi.org/10.1007/978-1-4842-1245-5

[2] Fedosejev, A. (2022). *React.js Essentials*. Packt Publishing. ISBN: 978-1783551620

[3] Cherny, B. (2021). *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media. ISBN: 978-1492037651

[4] Walls, C. (2022). *Spring in Action, Sixth Edition*. Manning Publications. ISBN: 978-1617297571

[5] Scarioni, C. (2021). *Pro Spring Security*. Apress. https://doi.org/10.1007/978-1-4842-5052-5

[6] Chong, F., & Carraro, G. (2020). "Architecture Strategies for Catching the Long Tail." *Microsoft Architecture Journal*. https://docs.microsoft.com/en-us/previous-versions/aa479069

[7] Ferraiolo, D., Kuhn, D. R., & Chandramouli, R. (2019). *Role-Based Access Control, Third Edition*. Artech House. ISBN: 978-1596931138

[8] Jones, M., Bradley, J., & Sakimura, N. (2015). "JSON Web Token (JWT)." *RFC 7519*. IETF. https://tools.ietf.org/html/rfc7519

[9] Fette, I., & Melnikov, A. (2011). "The WebSocket Protocol." *RFC 6455*. IETF. https://tools.ietf.org/html/rfc6455

[10] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN: 978-0201633610

[11] OWASP Foundation. (2023). "Password Storage Cheat Sheet." *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/cheats

---

12.2 Industry & Fitness Research

[12] Sharma, R., Kumar, A., & Singh, P. (2020). "User Experience Patterns in Fitness Mobile Applications." *International Journal of Human-Computer Interaction*, 36(4), 312-325.

[13] Chen, L., & Lee, M. (2019). "Factors Affecting Member Retention in Fitness Centers: A Machine Learning Approach." *Journal of Sport Management*, 33(5), 408-421.

[14] Rodriguez, M. (2021). "Multi-Tenancy Patterns for SaaS Applications." *IEEE Software*, 38(2), 65-72.

[15] Williams, J. (2022). "Microservices Architecture in Modern Fitness Applications." *Proceedings of the ACM Symposium on Cloud Computing*, 456-468.

12.3 Technical Documentation

[16] React Documentation. (2024). *React Official Documentation*. Meta Platforms, Inc. https://react.dev/

[17] Spring Framework. (2024). *Spring Boot Reference Documentation*. VMware. https://docs.spring.io/spring-boot/docs/current/reference/html/

[18] Oracle Corporation. (2024). *Oracle Database Administrator's Guide*. https://docs.oracle.com/en/database/

[19] JSON Web Tokens. (2024). *Introduction to JSON Web Tokens*. Auth0. https://jwt.io/introduction

[20] WebSocket API. (2024). *The WebSocket API (WebSockets)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

12.4 Security Standards

[21] OWASP. (2023). *OWASP Top Ten 2021*. OWASP Foundation. https://owasp.org/Top10/

[22] NIST. (2020). *Digital Identity Guidelines*. NIST Special Publication 800-63B. https://pages.nist.gov/800-63-3/sp800-63b.html

[23] W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

12.5 Framework & Library Documentation

[24] Spring Security. (2024). *Spring Security Reference*. https://docs.spring.io/spring-security/reference/

[25] Hibernate ORM. (2024). *Hibernate User Guide*. https://docs.jboss.org/hibernate/orm/current/userguide/html_single/

[26] Axios. (2024). *Axios Documentation*. https://axios-http.com/docs/intro

[27] Framer Motion. (2024). *Framer Motion Documentation*. https://www.framer.com/motion/

[28] Lombok Project. (2024). *Project Lombok Documentation*. https://projectlombok.org/features/

[29] React Router. (2024). *React Router Documentation*. https://reactrouter.com/

[30] Recharts. (2024). *Recharts - A composable charting library*. https://recharts.org/

12.6 Database & Infrastructure

[31] HikariCP. (2024). *HikariCP - A solid, high-performance JDBC connection pool*. https://github.com/brettwooldridge/HikariCP

[32] Oracle JDBC. (2024). *Oracle JDBC Driver Documentation*. https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/

12.7 Citation Format

This document follows IEEE citation style. All URLs were verified as accessible as of February 2026.

12.8 Acknowledgments

- React and Spring Boot communities for comprehensive documentation
- OWASP for security guidelines
- Academic researchers in fitness technology domain

Chapter 13: Appendices

Appendix A: System Installation Guide

13.0.1 A.1 Prerequisites

The following tools must be installed before setting up the Gym Management System locally:

Tool	Min. Version	Purpose
Node.js	18.x	Frontend runtime
npm	9.x	Package manager
Java JDK	17+	Backend runtime
Maven	3.8+	Java build tool
Oracle Database	19c+	Primary data store
Git	2.x	Version control

13.0.2 A.2 Frontend Setup

```bash
# Clone repository
git clone https://github.com/coded-with-aryan0426/gym-management-system-fullstack.git
cd gym-management-system-fullstack/frontend

# Install dependencies
npm install

# Configure environment
cp .env.example .env
# Edit .env with your API base URL

# Start development server
npm run dev
```

The frontend development server starts on `http://localhost:5173` by default.

13.0.3 A.3 Backend Setup

```bash
# Navigate to backend directory
cd gym-management-system-fullstack/backend

# Configure database in application.properties:
# spring.datasource.url=jdbc:oracle:thin:@localhost:1521:orcl
# spring.datasource.username=YOUR_DB_USERNAME
# spring.datasource.password=YOUR_DB_PASSWORD
```

```
# Build and run
./mvnw spring-boot:run
```

The backend API server starts on `http://localhost:8080` by default.

13.0.4 A.4 Environment Variables Reference

Variable	Description	Example
`VITE_API_BASE_URL`	Backend API URL	`http://localhost:8080/api`
`VITE_WS_URL`	WebSocket server URL	`ws://localhost:8080/ws`
`spring.datasource.url`	JDBC connection string	`jdbc:oracle:thin:@...`
`jwt.secret`	JWT signing secret (min 256-bit)	`[secure-random-string]`
`spring.mail.host`	SMTP host for email OTP	`smtp.gmail.com`

Appendix B: Database Schema Reference

13.0.5 B.1 Core Tables Summary

Table Name	Primary Key	Description
`users`	`user_id`	All user accounts across roles
`roles`	`role_id`	Role definitions (MEMBER, TRAINER, OWNER, ADMIN)
`gyms`	`gym_id`	Gym tenant records
`memberships`	`membership_id`	Active and historical memberships
`membership_packages`	`package_id`	Configured membership packages
`pt_sessions`	`session_id`	Personal training session bookings
`trainer_details`	`trainer_id`	Trainer profile extensions
`conversations`	`conversation_id`	Chat conversation threads
`messages`	`message_id`	Individual chat messages
`notifications`	`notification_id`	System notifications

13.0.6 B.2 Key Relationships

- Each `users` record is associated with one or more `roles` via a join table.
- Each `gym` has exactly one `owner` (foreign key to `users`).
- Each `membership` references one `membership_package` and one `user`.
- Each `pt_session` references one trainer `user`, one member `user`, and belongs to one `gym`.
- Each `conversation` contains zero or more `messages`, and involves two or more `users`.

Appendix C: API Endpoints Reference

13.0.7 C.1 Authentication Endpoints

Method	Endpoint	Description
POST	`/api/auth/register`	Register a new user
POST	`/api/auth/login`	Authenticate and receive JWT
POST	`/api/auth/refresh`	Refresh expired JWT
POST	`/api/auth/logout`	Invalidate session
POST	`/api/auth/verify-otp`	Verify 2FA OTP

13.0.8 C.2 Membership Endpoints

Method	Endpoint	Description
GET	`/api/memberships/packages`	List available packages
POST	`/api/memberships/request`	Submit membership purchase request
PUT	`/api/memberships/{id}/approve`	Owner approves membership
PUT	`/api/memberships/{id}/reject`	Owner rejects membership
GET	`/api/memberships/my`	Get current member’s membership

13.0.9 C.3 PT Session Endpoints

Method	Endpoint	Description
GET	`/api/sessions/trainers`	Browse available trainers
POST	`/api/sessions/book`	Book a PT session
PUT	`/api/sessions/{id}/complete`	Mark session as complete
PUT	`/api/sessions/{id}/cancel`	Cancel a booked session
POST	`/api/sessions/{id}/rate`	Submit session rating

Appendix D: Technology Dependencies

13.0.10 D.1 Frontend Dependencies (Selected)

Package	Version	Licence
react	18.x	MIT
react-dom	18.x	MIT
react-router-dom	6.x	MIT
axios	1.x	MIT
socket.io-client	4.x	MIT
framer-motion	10.x	MIT
recharts	2.x	MIT
lucide-react	latest	ISC
typescript	5.x	Apache-2.0
vite	5.x	MIT

13.0.11 D.2 Backend Dependencies (Selected)

Package	Version	Licence
spring-boot-starter-web	3.x	Apache-2.0
spring-boot-starter-security	3.x	Apache-2.0
spring-boot-starter-data-jpa	3.x	Apache-2.0
spring-boot-starter-websocket	3.x	Apache-2.0
jjwt-api	0.11.x	Apache-2.0
lombok	1.18.x	MIT
ojdbc8	21.x	Oracle
spring-boot-starter-mail	3.x	Apache-2.0